

Chapter 15 Bargaining

In the human world, we are used to the idea of bargaining, or negotiation. We bargain with our partners about who will tidy the house; we bargain with our children about how many stories they can read before bed; and politicians regularly bargain in situations that have life-or-death outcomes. The purpose of bargaining is to reach an agreement, and in particular, agreement in the presence of conflicting goals and preferences. In this chapter, we will see how bargaining, or negotiation, can be formalized and implemented.

15.1 Negotiation Parameters

We start by classifying the components that make up a negotiation setting.

- a *negotiation set*, which represents the space of possible proposals that agents can make

NEGOTIATION SET

- a *protocol*, which defines the legal proposals that agents can make, as a function of prior negotiation history
- a collection of *strategies*, one for each agent, which determine what proposals the agents will make. Usually, the strategy that an agent plays is *private*: the fact that an agent is using a particular strategy is not generally visible to other negotiation participants (although most negotiation settings are ‘open cry’, in the sense that the actual proposals that are made *are* seen by all participants)
- a rule that determines when a deal has been struck, and what this *agreement deal* is.

Negotiation usually proceeds in a series of rounds, with some proposal (possibly more than one) made at every round. The proposals that agents make are defined by their strategy, must be drawn from the negotiation set, and must be legal, as defined by the protocol. If agreement is reached, as defined by the agreement rule, then negotiation terminates with the agreement deal.

The first attribute that may complicate negotiation is where *multiple issues* are involved. An example of a single-issue negotiation scenario might be where two agents were negotiating only the price of a particular good for sale. In such a scenario, the preferences of the agents are symmetric, in that a deal which is more preferred from one agent’s point of view is guaranteed to be less preferred from the other’s point of view, and vice versa. Such symmetric scenarios are simple to analyse because it is always obvious what represents a concession: in order for the seller to concede, they must lower the price of their proposal, while for the buyer to concede, they must raise the price of their proposal. In *multiple-issue* negotiation scenarios, agents negotiate over not just the value of a single attribute, but the values of multiple attributes, which may be interrelated. For example, when buying a car, price is not the only issue to be negotiated (although it may be the dominant one). In addition, the buyer might be interested in the length of the guarantee, the terms of after-sales service, the extras that might be included such as air conditioning, stereos, and so on. In multiple-issue negotiations, it is usually much less obvious what represents a true concession: it is not simply the case that all attribute values must be either increased or decreased. (Salesmen in general, and car salesmen in particular, often exploit this fact during negotiation by making ‘concessions’ that are in fact no such thing.)

Multiple attributes also lead to an exponential growth in the space of possible deals. Let us take an example of a domain in which agents are negotiating over the value of n Boolean variables, $v_1, \dots, v_n$. A deal in such a setting consists of an assignment of either true or false to each variable v_i . Obviously, there are 2^n possible deals in such a domain. This means that, in attempting to decide what proposal to make next, it will be entirely unfeasible for an agent to explicitly consider every possible deal in domains of moderate size. Most negotiation domains are, of course, much more complex than this. For example, agents may need to negotiate about the value of attributes where these attributes can have m possible values, leading to a set of m^n possible deals. Worse, the objects of negotiation may be individually very complex indeed. In real-world negotiation settings – such as labour disputes or (to pick a rather extreme example) the kind of negotiation that was going on during the writing of the first edition of this book, with respect to the political future of Northern Ireland – there are not only many attributes, but the value of these attributes may be laws, procedures, and the like.

The negotiation participants may even have difficulty reaching agreement on what the attributes under negotiation actually are – a real-world example, again from Northern Ireland, was whether or not the decommissioning of paramilitary weapons should be up for negotiation. At times, it seemed that the different sides in this long-standing dispute had simultaneously had different beliefs about whether decommissioning was up for negotiation or not. Fortunately, in the end, they reached agreement – a happy example of successful negotiation! Another source of complexity in negotiation is the number of agents involved in the process, and the way in which these agents interact. There are three obvious possibilities.

One-to-one negotiation One agent negotiates with just one other agent. A particularly simple case of one-to-one negotiation is that where the agents involved have symmetric preferences with respect to the possible deals. An example from everyday life would be the type of negotiation we get involved in when discussing terms with a car salesman. We will see examples of such symmetric negotiation scenarios later.

Many-to-one negotiation In this setting, a single agent negotiates with a number of other agents. Auctions, as discussed above, are one example of many-to-one negotiation. For the purposes of analysis, many-to-one negotiations can often be treated as a number of concurrent one-to-one negotiations.

Many-to-many negotiation In this setting, many agents negotiate with many other agents simultaneously. In the worst case, where there are n agents involved in negotiation in total, this means that there can be up to $n(n-1)/2$ negotiation threads. Clearly, from an analysis point of view, this makes such negotiations hard to handle.

For these reasons, most attempts to automate the negotiation process have focused on rather simple settings. Single-issue, symmetric, one-to-one negotiation is the most commonly analysed, and we will focus on such settings here.

15.2 Bargaining for Resource Division

We begin by looking at one of the most influential general families of bargaining protocol: the *alternating offers* bargaining model of Rubinstein [Kraus, [2001](#); Osborne and Rubinstein, [1990](#)].

Alternating offers is a one-to-one protocol: we assume just two agents, agent 1 and agent 2. Negotiation takes place in a sequence of rounds, which we will assume are indexed by the natural numbers 0, 1, and so on. Agent 1 begins, at round 0, by making a proposal x^0 (from the negotiation set), which agent 2 can either accept (A) or reject (R). If the proposal is accepted, then the deal x^0 is implemented. Otherwise, if agent 2 rejected the proposal, then negotiation moves to another round, where agent 2 makes a proposal and agent 1 chooses to either accept or reject it. The overall protocol is illustrated by the state transition diagram in [Figure 15.1](#).

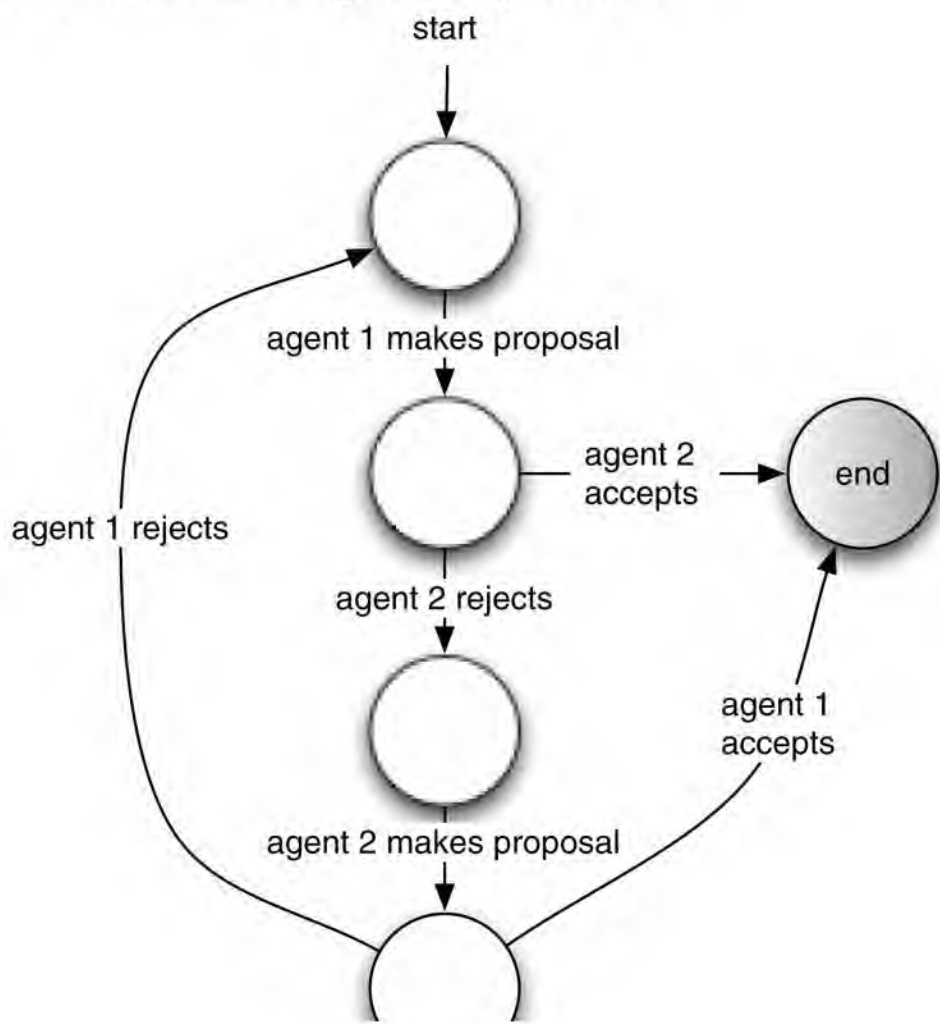
15.2.1 Patient players

There is of course nothing to stop negotiation using the alternating offers protocol going on for ever: according to the protocol, the agents can just keep on R -ing and R -ing and R -ing ... If the agents *never* reach agreement (i.e. a history of the proposal simply consists of a sequence of deals and associated R s), then we define the outcome of negotiation to be the *conflict deal*, Θ .

The following basic assumptions are made [Osborne and Rubinstein, 1990, pp. 33–34] (some other technical assumptions are made, but these are the central ones):

- Disagreement is the worst outcome** Both agents prefer any other outcome over disagreement.
- Agents seek to maximize utility** Agents really do prefer to get larger utility values.

Figure 15.1: The alternating offers protocol.



This basic model captures perhaps the most pertinent aspects of negotiation, but as it stands, it admits some arguably strange and undesirable behaviours. We will use the scenario of *dividing a pie* to illustrate alternating offers [Osborne and Rubinstein, 1990, p. 42].

The idea is that there is some resource whose value is ‘1’, which can be divided into two parts, such that (i) the values of the two parts must each be between 0 and 1; and (ii) the values of the parts sum to 1. Thus a *proposal* in this scenario is a pair $(x, 1 - x)$, where x is the amount of the pie that agent 1 gets, and $1 - x$ is the amount of the pie that agent 2 gets. The space of possible deals (the negotiation set) is thus the set:

$$\{(x, 1 - x) : 0 \leq x \leq 1\}.$$

Now, suppose you are agent 1, so that you get to make the first proposal. What should you do? To understand the answer to this question, we will first look at a few simpler cases. First, suppose that we *only allow one round* to take place. Thus, agent 1 makes a proposal; agent 2 can either accept it (in which case the deal is implemented), or else reject it (in which case the conflict deal Θ is implemented). In the literature, this is called an *ultimatum game*. In this case, it turns out that player 1 – the *first mover* – has all the power. Suppose that he proposes to get all the pie, i.e. proposes the deal $(1, 0)$. If agent 2 rejects, then the conflict deal is implemented; since by definition agent 2 would prefer to get 0 rather than the conflict deal, he accepts. Of course, getting the whole pie on the first round is obviously agent 1’s best outcome – he cannot improve on it at all. Thus these two strategies are in fact in Nash equilibrium.

ULTIMATUM GAE

Now consider the case where we permit *two* rounds of negotiation. Here, agent 1 makes a proposal, and agent 2 can either accept it or make a counter offer, which agent 1 can either accept or reject; if the offer is rejected at this stage, then the conflict deal Θ is implemented. Here, interestingly, it turns out that there is no advantage to going first: nothing agent 1 proposes can make any difference. By simply rejecting agent 1’s proposal, agent 2 turns negotiation into a one-round negotiation scenario, which, by the above argument, gives all the pie to the proposer, in this case, agent 2. Thus whatever agent 1 proposes, agent 2 rejects it and proposes $(0, 1)$, which agent 1 then accepts, because otherwise the least desirable outcome Θ would be implemented.

In general, if we set the number of permissible bargaining rounds to be some fixed value, then whoever moves last will get all the pie, by the argument above: the last round is an ultimatum game, and whoever makes the proposal on this last round has all the power.

Now let’s move on to the general case, where there is *no* bound on the number of rounds – negotiation can go on indefinitely. Suppose that agent 1 uses the following strategy:

Agent 1 always proposes that agent 1 gets the whole pie, and rejects any other offer.

What is agent 2’s best response to this? If agent 2 continually *rejects* the proposal, then the agents will never reach agreement, and so by definition the conflict deal Θ is enacted. By definition, this is the worst outcome for agent 2 – in particular, it is worse than the deal where

agent 1 gets the whole pie and agent 2 gets nothing. So agent 2 can do no better than *accepting* the proposal that agent 1 makes. This pair of strategies, where agent 1 proposes to get the whole pie and agent 2 accepts immediately, are in Nash equilibrium.

In fact, it is not hard to see, by essentially the same argument, that the alternating offers bargaining scheme admits an *infinite* set of Nash equilibrium outcomes: *for any possible deal $(x, 1 - x)$ in the negotiation set, there is a Nash equilibrium pair of negotiation strategies such that the outcome will be agreement on this deal in the first time period.* Notice that for agent 2 to ‘understand’ the situation, it must have access to agent 1’s strategy. In human terms, this would mean something like agent 2 being convinced that agent 1 was going to use this strategy. In terms of software agents, however, we can place a different and perhaps more realistic interpretation on this: namely, that agent 2 *has access to the program that implements agent 1’s negotiation.* (This is not so unrealistic: think of agent 1 as a mobile agent, or Java applet, for example.) In such cases, agent 1 can *benefit* by making its code available: it can in effect say ‘look, here is my strategy – you have no choice other than to accept my proposal’. (We will see this phenomenon again later in this chapter.)

15.2.2 Impatient players

This analysis seems to tell us that if no limits are placed on the length of time permitted for negotiation then the solution concept of Nash equilibrium is *too weak* – because there are an infinite number of Nash equilibria. However, if the model of alternating offers is altered somewhat, we can obtain an interesting result – a kind of ‘fundamental theorem’ of alternating offers. Given a modest set of additional assumptions, it is possible to prove that there is a *unique* solution (technically, a subgame perfect equilibrium). The extension of the framework that we consider adds the following assumption to the list of assumptions that we gave earlier.

Time is valuable For any outcome x and times t_1 and t_2 , both agents would prefer outcome x at time t_1 over outcome x at time t_2 if $t_2 > t_1$. Thus, given a choice between agreement x being made now and x being made later, our agents would always prefer x to be made now.

In other words, agents are, to varying degrees, *impatient*. Notice that if our agents have different preferences over time, then the *more patient* agent will benefit in negotiating, because it has increased power. It can ‘hold out’ for longer, and in particular, its *threats* will be more credible.

A standard approach to impatience in the alternating offers protocol is to use a *discount factor* δ_i for each agent ($0 \leq \delta_i < 1$). The idea is as follows. Suppose that agent i has a discount factor of δ_i , and at time 0 (i.e. in the first step of negotiation) i is offered a slice of the pie of size x (thus $0 \leq x \leq 1$), with the other agent getting $1 - x$. Now:

DISCOUNT FACTOR

- the value of the slice x at time 0 is $\delta_i^0 x = x$
- the value of the slice x at time 1 is $\delta_i^1 x = \delta_i x$
- the value of the slice x at time 2 is $\delta_i^2 x = (\delta_i \delta_i) x$

• ...

- the value of the slice x at time k is $\delta^k x$.

A *larger* value for δ_i (closer to 1) thus implies *more* patience; a *smaller* value means *less* patience. As δ_i tends to 1, then the agent becomes more and more indifferent to time, i.e. does not care at what time agreement is reached. In contrast, if $\delta_i = 0$, then agent i needs agreement in the first time step; the pie has no value to i at any later time point. So, we assume the two negotiating agents have discount factors δ_1 and δ_2 respectively.

To analyse the general case, let's first consider bargaining over fixed periods of time, as above. Consider the one-round case: here, the analysis is identical to the one-round case given above: we simply have an ultimatum game. If we allow *two* rounds, however, the situation changes somewhat, *because the value of the pie reduces in accordance with discount factors δ_i* . Suppose agent 2 rejected agent 1's initial proposal, whatever it was. Then the second round would be an ultimatum game, and agent 2 would be able to ensure getting the whole pie – it could make a proposal (0, 1) and agent 1 could do no better than accept this. But even though agent 2 is getting all the pie, the *value* of that pie has reduced: it is only worth δ_2 to agent 2. Agent 1 can take this fact into account in making his initial offer. Suppose he makes as the first proposal the deal $(1 - \delta_2, \delta_2)$. In this case, agent 2 may as well accept; if he rejects, then by the time he gets to make a counter offer, the value of the whole pie that he is able to ensure that he gets is reduced to δ_2 anyway. So, if agent 1 opens bargaining by proposing the deal $(1 - \delta_2, \delta_2)$, then agent 2 can do no better than accept: these strategies are in Nash equilibrium.

Now let us consider the general case, where there are no bounds on the number of rounds. Here, agent 1 (making the first proposal) makes the proposal that gives agent 2 what he would be able to enforce for himself in the second time period; and agent 2, knowing that he can do no better than this, accepts. Thus agreement will be reached in the first time step, with agent 1 getting

$$\frac{1 - \delta_2}{1 - \delta_1 \delta_2}$$

of the pie, and agent 2 getting the remainder, which works out to be

$$\frac{\delta_2(1 - \delta_1)}{1 - \delta_1 \delta_2}$$

of the pie. As δ_1 approaches the value 1 (i.e. as agent 1 becomes more and more patient) than its slice of the pie becomes larger and larger.

15.2.3 Negotiation decision functions

The analysis of alternating offers we have given above is very strategic: it assumes that both agents will try to maximize utility, and that they will assume that the other agent will do likewise. A simpler, non-strategic approach is to use *negotiation decision functions* [Faratin et al., 1998]. The idea is simply that agents use a time-dependent function to determine what

proposal they should make; this *negotiation decision function* only depends on the time to determine a proposal. It does not consider how other negotiation participants can or do behave.

NEGOTIATION DECISION FUNCTION

We will look at two classic examples of negotiation decision functions. Imagine that we are using the alternating offers protocol to sell a particular good. Clearly, the buyer wants the highest price possible, while the seller wants the lowest price possible. [Figures 15.2](#) and [15.3](#) give two classic negotiation decision functions for buyers and sellers in the alternating offers model. Consider [Figure 15.2](#), which shows the negotiation decision functions from the point of view of an agent who is selling some good, and hence desires a high price for it. In this strategy, the agent starts off by making an offer that represents a high price, and then over time (as it makes concessions) the price represented in its offers comes down. The shape of the graph is determined by the negotiation decision function. The difference between the two strategies – which are generally called *Boulware* and *Conceder* – lies in the way in which the price comes down. In the Boulware strategy, the agent initially does not decrease the price by very much at each time step; however, as its deadline for negotiation begins to approach, the size of its concessions increases by progressively larger and larger amounts, exponentially decaying to its *reserve price* for this transaction (indicated by a price of 0 on the graph). In the *conceder* strategy, however, the agent *makes all its concessions early* and then does not concede much as negotiation progresses and the deadline looms. These two strategies are illustrated from the point of view of a buyer in [Figure 15.3](#).

BOULWARE
CONCEDER

Figure 15.2: Two classic seller strategies for the bargaining protocol.

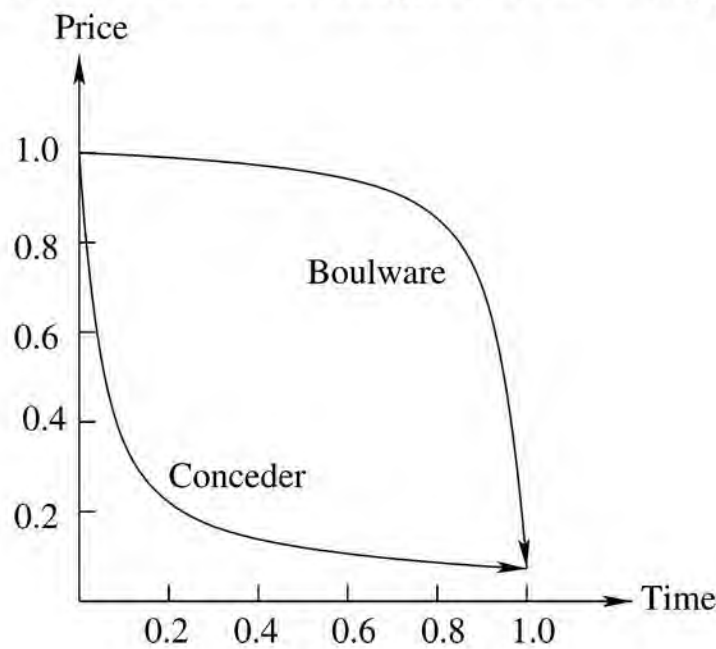
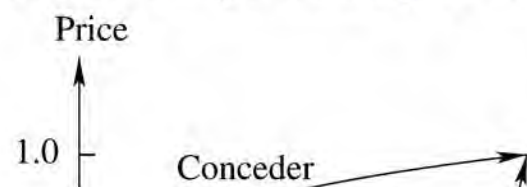
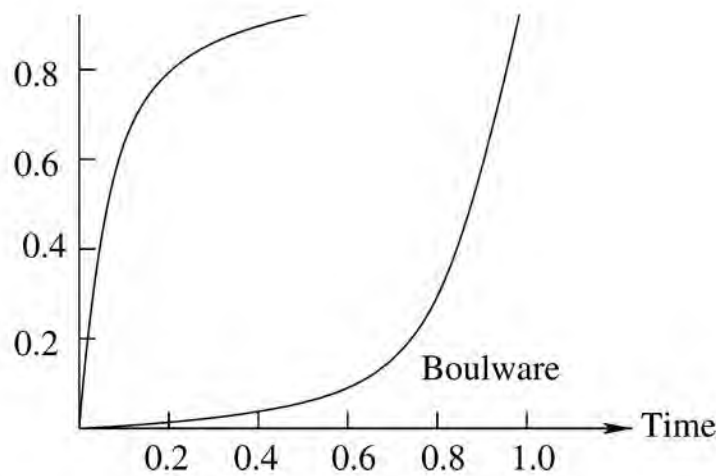


Figure 15.3: Two classic buyer strategies for the bargaining protocol.





15.2.4 Applications of alternating offers

The alternating offers model is one of the most popular and most influential models of bargaining in multiagent systems. For example, [Kraus, 2001] studies its use in the *data allocation problem*, where one is given a set of *servers* and a set of *datasets*, with utility functions for each server. The servers are each allocated a set of data sets, and provide answers to queries posted by a local client base. Answering a query may require only locally held datasets, or datasets held by other queries, in which case the datasets must be purchased. Negotiation in this domain relates to the way in which datasets are allocated to servers: suppose a server has queries relating to a particular dataset, then it may benefit from being allocated this dataset and not having to purchase it from other servers. But, if queries relating to the dataset are popular (think ‘Britney Spears’), then other servers might also prefer this allocation. Kraus gives a detailed analysis of the different kinds of utility functions that servers might have in such scenarios, and the different ways in which such utility functions can depend upon time.

[Kraus, 2001] also applies negotiation to the *resource allocation problem*, where the situation is as follows. The agents are negotiating over the use of some resource that, although it may be renewed infinitely often (in the sense that once one agent has finished using it, another agent can begin), cannot be used by more than one agent at the same time. A deal in such a scenario thus involves producing a schedule, describing who will have access to the resource when. The story is made more interesting by the fact that the agents may have preferences about when they have access to the resource.

15.3 Bargaining for Task Allocation

In this section, we consider negotiation for the *task-oriented domains* of [Rosenschein and Zlotkin, 1994, pp. 29–52]. The idea here is simply that agents who have tasks to carry out may be able to benefit by reorganizing the distribution of tasks among themselves; but this raises the issue of how to reach agreement on who will do which tasks. Consider the following example.

TASK-ORIENTED DOMAIN

Imagine that you have three children, each of whom needs to be delivered to a different school each morning. Your neighbour has four children, and also needs to take them to school. Delivery of each child can be modelled as an indivisible task. You and your neighbour can discuss the situation, and come to an agreement that it is better for both of you

(for example, by carrying the other's child to a shared destination, saving him the trip). There is no concern about being able to achieve your task by yourself. The worst that can happen is that you and your neighbour will not come to an agreement about setting up a car pool, in which case you are no worse off than if you were alone. You can only benefit (or do no worse) from your neighbour's tasks.

Assume, though, that one of my children and one of my neighbours' children both go to the same school (that is, the cost of carrying out these two deliveries, or two tasks, is the same as the cost of carrying out one of them). It obviously makes sense for both children to be taken together, and only my neighbour or I will need to make the trip to carry out both tasks.

What kinds of agreement might we reach? We might decide that I will take the children on even days each month, and my neighbour will take them on odd days; perhaps, if there are other children involved, we might have my neighbour always take those two specific children, while I am responsible for the rest of the children.

[Rosenschein and Zlotkin, 1994, p. 29]

To formalize this kind of situation, Rosenschein and Zlotkin defined the notion of a *task-oriented domain* (TOD). A task-oriented domain is a triple

$$(T, Ag, c),$$

where

- T is the (finite) set of all possible tasks
- $Ag = \{1, \dots, n\}$ is the (finite) set of negotiation participant agents
- $c: 2^T \rightarrow \mathbb{R}_+$ is a function which defines the *cost* of executing each subset of tasks: the cost of executing any set of tasks is a positive real number.

The cost function must satisfy two constraints. First, it must be *monotonic*. Intuitively, this means that adding tasks never decreases the cost. Formally, this constraint is defined as follows:

$$\text{If } T_1, T_2 \subseteq T \text{ are sets of tasks such that } T_1 \subseteq T_2, \text{ then } c(T_1) \leq c(T_2).$$

The second constraint is that the cost of doing nothing is zero, i.e. $c(\emptyset) = 0$.

An *encounter* within a task-oriented domain (T, Ag, c) occurs when the agents Ag are assigned tasks to perform from the set T . Intuitively, when an encounter occurs, there is potential for the agents to reach a deal by reallocating the tasks among themselves; as we saw in the informal car pool example above, by reallocating the tasks, the agents can potentially do better than if they simply performed their tasks themselves. Formally, an encounter in a TOD (T, Ag, c) is a collection of tasks

ENCOUNTER

$$(T_1, \dots, T_n),$$

where, for all i , we have that $i \in Ag$ and $T_i \subseteq T$. Notice that a TOD together with an encounter in this TOD is a type of *task environment* of the kind we saw in [Chapter 2](#). It defines both the characteristics of the environment in which the agent must operate, together with a task (or rather, set of tasks), which the agent must carry out in the environment.

Hereafter, we will restrict our attention to one-to-one negotiation scenarios, as discussed above:

Hereafter, we will restrict our attention to one-to-one negotiation scenarios, as discussed above. we will assume that the two agents in question are $\{1, 2\}$. Now, given an encounter (T_1, T_2) , a *deal* will be very similar to an encounter: it will be an allocation of the tasks $T_1 \cup T_2$ to the agents 1 and 2. Formally, a pure deal is a pair (D_1, D_2) where $D_1 \cup D_2 = T_1 \cup T_2$. The semantics of a deal (D_1, D_2) is that agent 1 is committed to performing tasks D_1 and agent 2 is committed to performing tasks D_2 .

The *cost* to agent i of a deal $\delta = (D_1, D_2)$ is defined to be $c(D_i)$, and will be denoted $cost_i(\delta)$. The *utility* of a deal δ to an agent i is the difference between the cost of agent i doing the tasks T_i that it was originally assigned in the encounter, and the cost $cost_i(\delta)$ of the tasks it is assigned in δ :

$$utility_i(\delta) = c(T_i) - cost_i(\delta).$$

Thus the utility of a deal represents how much the agent has to gain from the deal; if the utility is negative, then the agent is *worse off* than if it simply performed the tasks it was originally allocated in the encounter.

What happens if the agents *fail* to reach agreement? In this case, they must perform the tasks (T_1, T_2) that they were originally allocated. This is the intuition behind the terminology that the *conflict deal*, denoted Θ , is the deal (T_1, T_2) consisting of the tasks originally allocated.

CONFLICT DEAL

The notion of *dominance*, as discussed in the preceding chapter, can be easily extended to deals. A deal δ_1 is said to dominate deal δ_2 (written $\delta_1 \succ \delta_2$) if and only if the following hold.

1. Deal δ_1 is at least as good for every agent as δ_2 :

$$\forall i \in \{1, 2\}, utility_i(\delta_1) \geq utility_i(\delta_2).$$

2. Deal δ_1 is better for some agent than δ_2 :

$$\exists i \in \{1, 2\}, utility_i(\delta_1) > utility_i(\delta_2).$$

If deal δ_1 dominates another deal δ_2 , then it should be clear to all participants that δ_1 is better than δ_2 . That is, all ‘reasonable’ participants would prefer δ_1 to δ_2 . Deal δ_1 is said to weakly dominate δ_2 (written $\delta_1 \succeq \delta_2$) if at least the first condition holds.

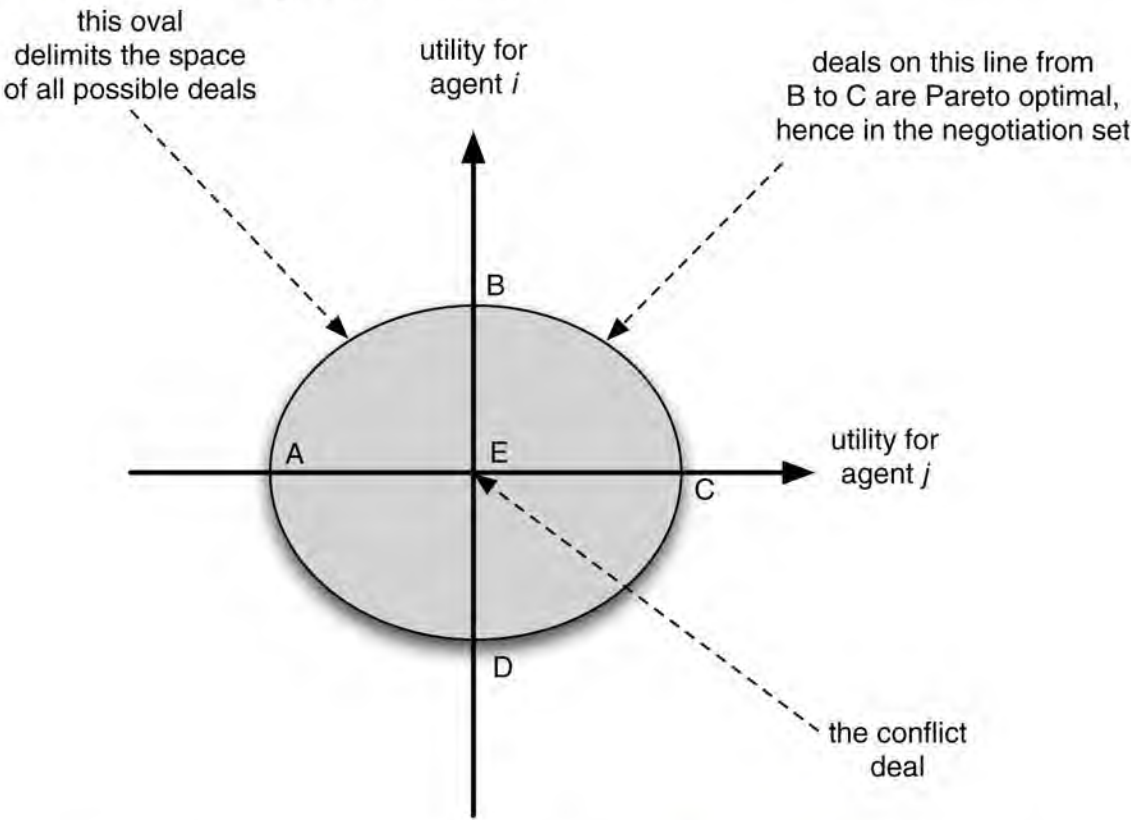
A deal that is not dominated by any other deal is said to be *Pareto optimal*. Formally, a deal δ is Pareto optimal if there is no deal δ' such that $\delta' \succ \delta$. If a deal is Pareto optimal, then there is no alternative deal that will improve the lot of one agent except at some cost to another agent (who presumably would not be happy about it!). If a deal is not Pareto optimal, however, then the agents could improve the lot of at least one agent, without making anyone else worse off.

A deal δ is said to be *individual rational* if it weakly dominates the conflict deal. If a deal is *not* individual rational, then at least one agent can do better by simply performing the tasks it was originally allocated – hence it will prefer the conflict deal. Formally, deal δ is individual rational if and only if $\delta \succeq \Theta$.

We are now in a position to define the space of possible proposals that agents can make. The negotiation set consists of the set of deals that are (i) individual rational, and (ii) Pareto

negotiation set consists of the set of deals that are (i) individual rational, and (ii) Pareto optimal. The intuition behind the first constraint is that there is no purpose in proposing a deal that is less preferable to some agent than the conflict deal (as this agent would prefer conflict); the intuition behind the second condition is that there is no point in making a proposal if an alternative proposal could make some agent better off at nobody's expense.

Figure 15.4: The negotiation set.



The intuition behind the negotiation set is illustrated in [Figure 15.4](#). In this graph, the space of all conceivable deals is plotted as points on a graph, with the utility to i on the y -axis, and utility to j on the x -axis. The shaded space enclosed by points A, B, C, and D contains the space of all possible deals. The conflict deal is marked at point E. It follows that all deals to the left of the line B–D will not be individual rational for agent j (because j could do better with the conflict deal). For the same reason, all deals below line A–C will not be individual rational for agent i . This means that the negotiation set contains deals in the shaded area B–C–E. However, not all deals in this space will be Pareto optimal. In fact, the only Pareto optimal deals that are also individual rational for both agents will lie on the line B–C. Thus the deals that lie on this line are those in the negotiation set. Typically, agent i will start negotiation by proposing the deal at point B, and agent j will start by proposing the deal at point C.

15.3.1 The monotonic concession protocol

MONOTONIC CONCESSION PROTOCOL

The protocol we will introduce for this scenario is known as the *monotonic concession protocol* [Rosenschein and Zlotkin, 1994, pp. 40–41]. The rules of this protocol are as follows.

- Negotiation proceeds in a series of rounds.
- On the first round, both agents simultaneously propose a deal from the negotiation set.
- An agreement is reached if the two agents propose deals δ_1 and δ_2 , respectively, such that either (i) $utility_i(\delta_1) > utility_i(\delta_2)$ or (ii) $utility_j(\delta_1) > utility_j(\delta_2)$ i.e. if one of the

that either (i) $utility_1(o_2) \geq utility_1(o_1)$ or (ii) $utility_2(o_1) \geq utility_2(o_2)$, i.e. if one of the agents finds that the deal proposed by the other is at least as good as or better than the proposal it made.

If agreement is reached, then the rule for determining the agreement deal is as follows. If both agents' offers match or exceed those of the other agent, then one of the proposals is selected at random. If only one proposal exceeds or matches the other's proposal, then this is the agreement deal.

- If no agreement is reached, then negotiation proceeds to another round of simultaneous proposals. In round $u + 1$, no agent is allowed to make a proposal that is less preferred by the other agent than the deal it proposed at time u .
- If neither agent makes a concession in some round $u > 0$, then negotiation terminates, with the conflict deal.

It should be clear that this protocol is effectively *verifiable*: it is easy for both parties to see that the rules of the protocol are being adhered to.

Using the monotonic concession protocol, negotiation is guaranteed to end (with or without agreement) after a finite number of rounds. Since the set of possible deals is finite, the agents cannot negotiate indefinitely: either the agents will reach agreement, or a round will occur in which neither agent concedes. However, the protocol does not guarantee that agreement will be reached *quickly*. Since the number of possible deals is $O(2^{|T|})$, it is conceivable that negotiation will continue for a number of rounds exponential in the number of tasks to be allocated.

15.3.2 The Zeuthen strategy

So far, we have said nothing about how negotiation participants might or should behave when using the monotonic concession protocol. On examining the protocol, it seems that there are three key questions to be answered:

- What should an agent's first proposal be?
- On any given round, *who should concede*?
- If an agent concedes, then *how much* should it concede?

The first question is straightforward enough to answer: an agent's first proposal should be its most preferred deal.

With respect to the second question, the idea of the Zeuthen strategy is to measure an agent's *willingness to risk conflict*. Intuitively, an agent will be more willing to risk conflict if the difference in utility between its current proposal and the conflict deal is low.

WILLINGNESS TO RISK CONFLICT

In contrast, if the difference between the agent's current proposal and the conflict deal is high, then the agent has more to lose from conflict and is therefore less willing to risk conflict – and thus should be more willing to concede.

Agent i 's willingness to risk conflict at round t , denoted $risk_i^t$, is measured in the following way [Rosenschein and Zlotkin, 1994, p. 43]:

$$risk_i^t = \frac{\text{utility } i \text{ loses by conceding and accepting } j \text{'s offer}}{\text{utility } i \text{ loses by not conceding and causing conflict}}.$$

The numerator on the right-hand side of this equation is defined to be the difference between the utility to i of its current proposal, and the utility to i of j 's current proposal; the denominator is defined to be the utility of agent i 's current proposal. Until an agreement is reached, the value of $risk_i^t$ will be a value between 0 and 1. Higher values of $risk_i^t$ (nearer to 1) indicate that i has less to lose from conflict, and so is more willing to risk conflict.

Conversely, lower values of $risk_i^t$ (nearer to 0) indicate that i has more to lose from conflict, and so is less willing to risk conflict.

Formally, we have

$$risk_i^t = \begin{cases} 1 & \text{if } utility_i(\delta_i^t) = 0, \\ \frac{utility_i(\delta_i^t) - utility_i(\delta_j^t)}{utility_i(\delta_i^t)} & \text{otherwise.} \end{cases}$$

The idea of assigning risk the value 1 if $utility_i(\delta_i^t) = 0$ is that in this case, the utility to i of its current proposal is the same as from the conflict deal; in this case, i is completely willing to risk conflict by not conceding.

ZEUTHEN STRATEGY

So, the Zeuthen strategy proposes that the agent to concede on round t of negotiation should be the one with the smaller value of risk.

The next question to answer is *how much should be conceded?* The simple answer to this question is *just enough*. If an agent does not concede enough, then on the next round, the balance of risk will indicate that it still has most to lose from conflict, and so should concede again. This is clearly inefficient. On the other hand, if an agent concedes too much, then it 'wastes' some of its utility. Thus an agent should make the *smallest* concession necessary to change the balance of risk – so that on the next round, the other agent will concede.

There is one final refinement that must be made to the strategy. Suppose that, on the final round of negotiation, both agents have *equal* risk. Hence, according to the strategy, both should concede. But, knowing this, one agent can 'defect' (cf. discussions in the preceding chapter) by not conceding, and so benefit from the other. If both agents behave in this way, then conflict will arise, and no deal will be struck. We extend the strategy by an agent 'flipping a coin' to decide who should concede if ever an equal risk situation is reached on the last negotiation step.

Now, given the protocol and the associated strategy, to what extent does it satisfy the desirable criteria for mechanisms discussed at the opening of this chapter? While the protocol does not guarantee success, it does guarantee termination; it does not guarantee to maximize social welfare, but it does guarantee that if agreement is reached, then this agreement will be

Pareto optimal; it is individually rational (if agreement is reached, then this agreement will be better for both agents than the default, conflict deal); and clearly there is no single point of failure – it does not require a central arbiter to monitor negotiation. With respect to simplicity and stability, a few more words are necessary. As we noted above, the space of possible deals may be exponential in the number of tasks allocated. For example, in order to execute his strategy, an agent may need to carry out $O(2^{|T|})$ computations of the cost function [Rosenschein and Zlotkin, 1994, p. 49]. This is clearly not going to be feasible in practice for any realistic number of tasks.

With respect to stability, we here note that the Zeuthen strategy (with the equal risk rule) is in Nash equilibrium, as discussed in the previous chapter. Thus, under the assumption that one agent is using the strategy, the other can do no better than use it himself.

This is of particular interest to the designer of automated agents. It does away with any need for secrecy on the part of the programmer. An agent's strategy can be publicly known, and no other agent designer can exploit the information by choosing a different strategy. In fact, it is desirable that the strategy be known, to avoid inadvertent conflicts.

[Rosenschein and Zlotkin, 1994, p. 46]

15.3.3 Deception

An interesting issue arises when one considers that agents need not necessarily be truthful when declaring their tasks in an encounter. By so doing, they can subvert the negotiation process. There are two obvious ways in which an agent can be deceitful in such domains, as follows.

Phantom and decoy tasks Perhaps the most obvious way in which an agent can deceive for personal advantage in task-oriented domains is by pretending to have been allocated a task that it has not been allocated. These are called *phantom* tasks. Returning to the car pool example, above, one might pretend that some additional task was necessary by saying that one had to collect a relative from a train station, or visit the doctor, at the time when the children needed to be delivered to school. In this way, the apparent structure of the encounter is changed, so that the outcome is in favour of the deceitful agent. The obvious response to this is to ensure that the tasks an agent has been assigned to carry out are *verifiable* by all negotiation participants. In some circumstances, it is possible for an agent to produce an artificial task when asked for it. Detection of such *decoy* tasks is essentially impossible, making it hard to be sure that deception will not occur in such domains. Whether or not introducing artificial tasks is beneficial to an agent will depend on the particular TOD in question.

PHANTOM TASKS

DECOY TASKS

Hidden tasks Perhaps counterintuitively, it is possible for an agent to benefit from deception by *hiding* tasks that it has to perform. Again with respect to the car pool example, agent 1 might have two children to take to schools that are close to one another. It takes one hour for the agent to visit both schools, but only 45 minutes to visit just one. If the neighbour, agent 2, has to take a child to one of these schools, then by

hiding his task of going to one of these schools, agent 1 can perhaps get agent 2 to take his child, thus improving his overall utility slightly.

HIDDEN TASKS

Before we leave task-oriented domains, there are some final comments worth making. First, the attractiveness of the monotonic concession protocol and Zeuthen strategy is obvious. They closely mirror the way in which human negotiation seems to work – the assessment of risk in particular is appealing. The Nash equilibrium status of the (extended) Zeuthen strategy is also attractive. However, the computational complexity of the approach is a drawback. Moreover, extensions to $n > 2$ agent negotiation scenarios are not obvious – for the reasons discussed earlier, the technique works best with symmetric preferences. Nevertheless, variations of the monotonic concession protocol are in wide-scale use, and the simplicity of the protocol means that many variations on it have been developed.

15.4 Bargaining for Resource Allocation

The final topic we consider is somewhat related to the issue of resource allocation considered in [Chapter 14](#). Specifically, we consider the problem of how agents can *reallocate resources among one another* to mutual benefit [Sandholm, 1998]. The main difference from the auction mechanisms considered in [Chapter 14](#) is that in the auction settings we considered, we primarily focused on the allocation of a collection of resources from a single auctioneer to a group of potential bidders (i.e. the ‘negotiation’ was ‘one–many’). In contrast, the setting here is that of multiple agents, *each of which* has resources that they consider reallocating (i.e. negotiation is ‘many–many’).

The scenario that we are concerned with is encapsulated in the following definition. A *resource allocation setting* is defined by a tuple:

$$(Ag, Z, v_1, \dots, v_n)$$

RESOURCE ALLOCATION SETTING

where:

- $Ag = \{1, \dots, n\}$ as usual, is the set of agents
- $Z = \{z_1, z_2, \dots, z_m\}$ is the set of resources
- $v_i: 2^Z \rightarrow \mathbb{R}$ is a *valuation function* for each agent $i \in Ag$.

Notice that valuation functions are here defined in exactly the same way as they were defined for combinatorial auctions in [Chapter 14](#). As in [Chapter 14](#), an allocation of Z to Ag is a partition $Z_1, Z_2, \dots, Z_n$ of the resources Z over the agents Ag .

Now, starting from some initial allocation $P_0 = Z_1^0, Z_2^0, \dots, Z_n^0$ agents can bargain with each other in an attempt to improve the value of their holding. If I have a resource that I do not value highly but you do, and you have a resource that you do not value highly but I do, then by exchanging resources, we can mutually benefit. A number of interpretations have been proposed in order to define what constitutes a reasonable transfer of resource both from an individual agent’s viewpoint and from the perspective of the overall allocation. Thus in

negotiating a change from an allocation P_i to Q_i (with $P_i, Q_i \subseteq Z$ and $P_i \neq Q_i$) there are three possible outcomes for the agent i :

$$v_i(P_i) < v_i(Q_i) \quad i \text{ is better off after the exchange}$$

$$v_i(P_i) = v_i(Q_i) \quad i \text{ is indifferent between } P_i \text{ and } Q_i$$

$$v_i(P_i) > v_i(Q_i) \quad i \text{ is worse off after the exchange.}$$

Let us dig a little deeper into the notion of an exchange. Suppose i has some good z_1 that it values at, say, 5 (i.e. $v_i(\{z_1\}) = 5$), but which j values at 10 ($v_j(\{z_1\}) = 10$). How can j persuade i to transfer the item? We assume that agents can make *side payments*. In order to accept the new allocation, i can be given some payment, sufficient to compensate for the resulting loss in utility. For example, j could give i a side payment of 5, ensuring that i is no worse off after the transfer; j is able to make this payment because by obtaining z_1 , it gets some additional value that it can share; even after making a payment of 5, agent j is still better off after the transaction than it was before. Ultimately, any side payment made must be funded by value received by some agent from their allocation.

SIDE PAYMENTS

Formally, we think of each agent i as making a payment, p_i : if $p_i < 0$ then in fact i is given $-p_i$ in return for accepting a contract; if $p_i > 0$ then i contributes p_i to the amount to be distributed among those agents whose pay-off is negative. Formally, a *pay-off vector*

$\bar{p} = (p_1, p_2, \dots, p_n)$ is a tuple of side payments, one to each agent, such that the total amount paid out equals the total amount received:

PAY-OFF VECTOR

$$\sum_{i=1}^n p_i = 0.$$

A *deal* is a triple

$$(Z, Z', \bar{p})$$

where $Z \in \text{alloc}(Z, \text{Ag})$ and $Z' \in \text{alloc}(Z, \text{Ag})$ are distinct allocations of Z to agents Ag , and $\bar{p}$ is a pay-off vector. We use δ to denote an arbitrary deal. The effect of implementing the deal $(Z, Z', \bar{p})$ is that the allocation of resources specified by Z is replaced with that specified by Z' , and payments as defined in $\bar{p}$ are made.

A deal $(Z, Z', \bar{p})$ is said to be *individually rational* if, taking into account both payments made and the allocation of resources that i receives from the deal, every agent i is *better off* if the deal was implemented with the side payments $\bar{p}$. Formally, individual rationality requires that

$$v_i(Z'_i) - p_i > v_i(Z_i)$$

for each agent i , except that p_i is allowed to be 0 if $Z_i = Z'_i$, (i.e. if the deal (Z, Z') leaves agent i with no change in its resources, then it is not required that i be rewarded). If a deal $(Z, Z', \bar{p})$ is not individually rational, then some agent would be worse off if Z' were implemented, compared to Z .

We use the notions of Pareto optimality and social welfare as defined in [Chapters 11](#) and [14](#): thus an allocation Z is Pareto optimal if every other allocation Z' that makes some agent strictly better off makes some other agent strictly worse off.

Protocol for resource reallocation

Now, given this framework, let us consider how agents can reallocate resources through negotiation. First, we must define a termination condition: a rule that defines when negotiation ends. The possible rules we consider are that termination ends when an allocation is reached that is either Pareto optimal or that maximizes social welfare. Given this rule, the protocol we consider is as follows:

1. We start off with the initial allocation, Z^0 : this is the initial allocation of resources to agents.
2. We define the *current allocation* to be Z_0 with 0 side payments (i.e. nobody makes or receives any side payments).
3. Any agent is permitted to put forward a deal $(Z, Z', \bar{p})$ as a proposal, where Z is the current allocation. (To realize this, imagine we continually select an agent at random and ask whether they want to make a proposal.)
4. If all agents agree to that deal (i.e. are satisfied with the allocation Z' and payments $\bar{p}$), and the relevant termination condition has been satisfied (e.g. the allocation maximizes social welfare, or is Pareto optimal, as appropriate), then negotiation terminates and the deal Z' is implemented with payments $\bar{p}$.
5. If every agent agrees to the deal but the termination condition is not reached, then the current allocation is set to Z' with payments $\bar{p}$, and we go to step 3.
6. If some agent is unsatisfied with the deal, then the current allocation remains unchanged, and we go to step 3.

(As an aside, notice that because of the way this protocol is formulated, we are not guaranteed to terminate. We could get stuck at step 3, if no agent wishes to make a proposal, or in the final step, if every agent vetoes every proposal. We won't worry about these issues here.)

Given this protocol, we can start to classify the kinds of deals that agents may propose. The main issue here is with respect to the computational complexity of making a proposal. The problem is that finding an optimal deal – one that maximizes social welfare, for example – is NP-hard. So, starting from some arbitrary allocation Z^0 , finding an optimal allocation immediately, in one round of bargaining, is going to be very hard. Instead, we can consider various restricted, computationally easy classes of deals.

O-contracts A *One-contract* (O-contract) is a deal that *only involves moving one resource and only involves one side payment* [Sandholm, [1998](#)]. The point about

O-contracts is that they greatly restrict the search space that must be considered when identifying possible proposal deals: if there are n agents in the system, then agent i only needs to consider $|Z_i|(n - 1)$ possible deals. Now, the good news about O-contracts is this: *starting from any arbitrary allocation, it is possible to reach an allocation that maximizes social welfare through a sequence of O-contracts*. To put it another way, with respect to the aim of finding allocations that maximize social welfare, we can restrict ourselves to O-contracts safe in the knowledge that we will be able to reach a socially optimal outcome. The bad news, however, is that sometimes the only possible way to get to a socially optimal outcome using O-contracts would involve agents accepting a proposal that is *not* individually rational. Now, clearly, accepting a proposal that is not individually rational, in the hope of getting a better deal later on, would involve a significant leap of faith on the part of negotiation participants. If the deal in question led to the best possible allocation for some particular agent, then he could veto any other subsequent proposals. So, while O-contracts can always lead to a socially optimal outcome in theory, they cannot always do so in practice.

O-CONTRACTS

C-contracts *Cluster contracts* (C-contracts) involve the transfer of *any number of resources greater than 1*, but again with the restriction that the deal involves an exchange from only one agent to one other (i.e. no multi-way exchanges are permitted). Thus a C-contract involves one agent transferring a subset of its allocation to another agent without receiving any subset of resources in return. The requirement in C-contracts that the transfer involves *more than one* resource in fact means that there are optimal allocations that cannot be reached through a series of C-contracts, individually rational or otherwise. (It is easy to construct an example, using just one resource.)

C-CONTRACTS

S-contracts *Swap contracts* (S-contracts) embody the very natural idea of agents simply swapping a resource (and of course making side payments). Thus an S-contract only involves two agents. Unfortunately, as with C-contracts, there are situations where no sequence of S-contracts (individual rational or otherwise) will lead to an optimal allocation.

S-CONTRACTS

M-contracts *Multiagent contracts* (M-contracts) involve *at least three* agents each transferring *a single* resource. Once again, there are situations in which no sequence of M-contracts will result in an optimal allocation.

M-CONTRACTS

So, for a natural collection of contract types, there will always be situations where no sequence of individually rational deals of the relevant type will result in an optimal allocation. This sounds like bad news. Moreover, for several of the types of contract listed above, determining whether a particular allocation can be reached by a series of individually rational

proposals is NP-hard [Dunne et al., [2005](#)].

Let us return to the original formulation of a deal, where we placed no restrictions on the types of transfers made. Suppose we place no restrictions on deals *other* than the fact that each deal *must* be individually rational: that is, each successive deal must strictly increase the overall pay-off of any agent that does not change its allocation. We obtain the following fact: all we have to do to guarantee that we will eventually reach an optimal outcome is to keep on proposing individually rational deals; eventually we will be guaranteed to reach an optimal allocation. The nice thing about this result is that we obtain a *globally* good outcome by using only *local* (one-step) reasoning: to ‘progress’, it is sufficient to simply find a proposal that is individually rational:

[A]gents can accept individually rational contracts as they are offered. They need not wait for more profitable ones, and they need not worry that a current contract may make a more profitable future contract unprofitable. Neither do they need to accept contracts that are not individually rational in anticipation of future contracts that make the combination beneficial.

[Sandholm, [1998](#)]

Another attractive feature is that the protocol guarantees that the quality of allocations proposed at each time step will monotonically increase. In the worst case, however, reaching an optimal allocation via a sequence of individually rational deals may require a number of proposals exponential in the number of agents and resources [Dunne, [2005](#)].

Notes and Further Reading

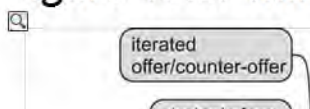
John Nash – of Nash equilibrium fame – was one of the first to study bargaining from a formal, game-theoretic viewpoint. His approach was rather different from the approaches studied here. He proposed a number of properties that a solution to a bargaining problem should have, and proved that these properties have a unique solution: now called the *Nash solution* [Osborne and Rubinstein, [1994](#), pp. 299–312]. The alternating offers protocol is due to Ariel Rubinstein [Rubinstein, [1982](#)]; the textbook by Osborne and Rubinstein is a good starting place for understanding bargaining theory [Osborne and Rubinstein, [1990](#)].

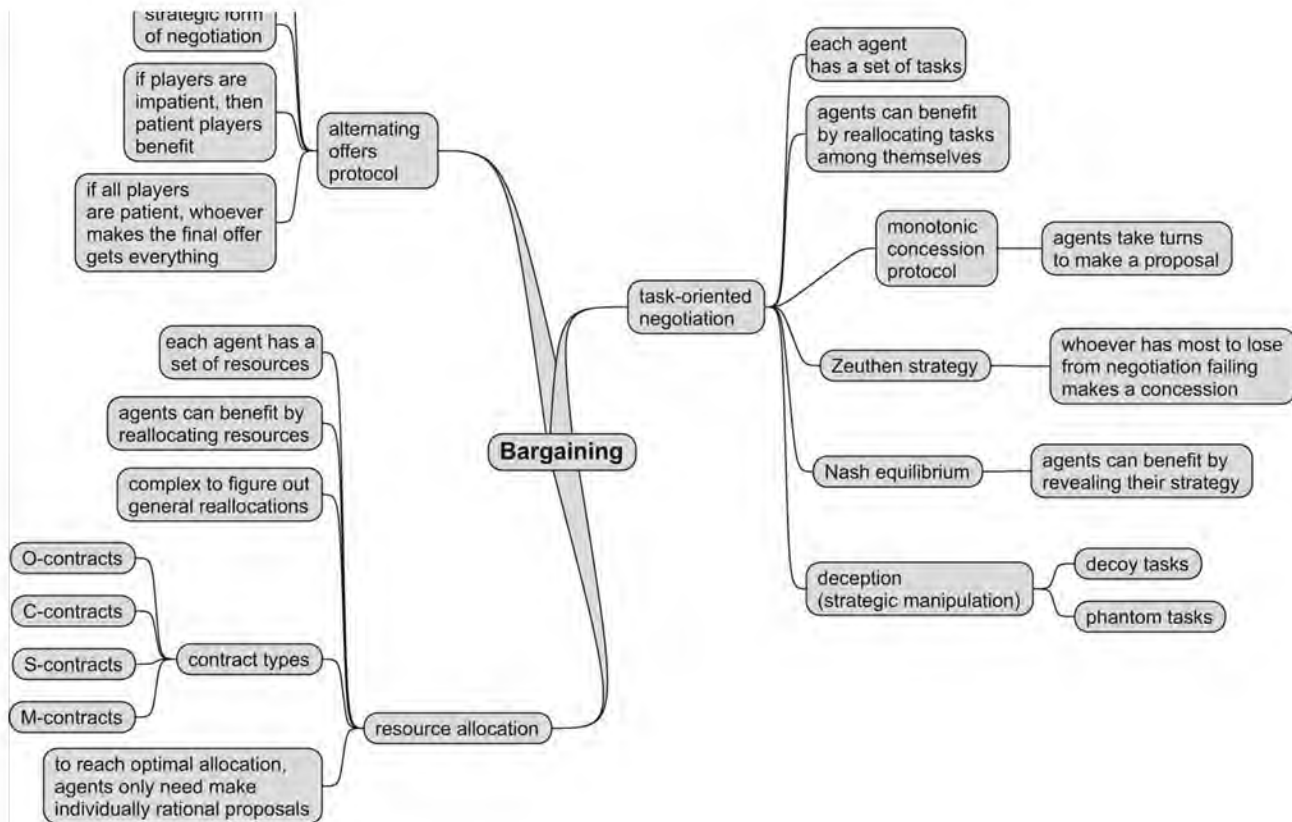
Bargaining in multiagent systems was introduced largely through the work of Sarit Kraus [Kraus and Wilkenfeld, [1991](#)] and Jeff Rosenschein and his students [Rosenstein and Zlotkin, [1994](#)]; the latter is an excellent starting place for reading on bargaining in multiagent systems. The work of Sarit Kraus builds on the alternating offers protocol, while Rosenschein’s starting point was Harsanyi’s bargaining model [Harsanyi, [1977](#)].

The bargaining framework for resource reallocation was originally presented in [Sandholm, [1998](#)], and was refined and studied in [Dunne et al., [2005](#); Endriss et al., [2003](#)].

Class reading: [Kraus, [1997](#)]. This article provides an overview of negotiation techniques for multiagent systems. It provides a number of pointers into the research literature, and will be particularly useful for mathematically oriented students.

Figure 15.5: Mind map for this chapter.





Chapter 16 Arguing

We have now seen a number of techniques through which agents can automatically reach agreements with one another on matters of common interest. In this chapter, we deal with a related problem, which we might understand as coming to agreement about *what to believe*. To better understand what we mean by this, consider the dialogues that take place in a court of law. The proceedings in a court of law are aimed at establishing a *rationally justifiable position* with respect to various *arguments* that are put forward. In a criminal court, the positions will typically relate to the guilt or innocence of the defendant. Now, if all the evidence and arguments put forward are mutually consistent, then establishing a position is easy, since there is basically no disagreement. This is not usually the case, however. Often, the evidence will be inherently contradictory; establishing a rationally justifiable position in such circumstances inherently means rejecting (or at least disregarding) some of the arguments put forward. Implicitly, then, argumentation involves dealing with inconsistencies among the beliefs of multiple agents. Sometimes, of course inconsistencies are obvious:

RATIONALLY JUSTIFIABLE POSITION

I believe p ; you believe $\neg p$.

Sometimes, however, the inconsistency is implicit:

I believe p , $p \rightarrow q$; you believe $\neg q$.

Note that when there is no inconsistency, there is no need for argumentation – we can just pool our positions together.

Thus one way of thinking about argumentation is as providing principled techniques for handling inconsistency, and in particular, for extracting rationally justifiable positions from the inconsistent pool of arguments. As we will see, one difficulty with this idea is that there can be *many* rational positions, in which case how do we define or find the ‘best’ such position? For example, if I believe p and you believe, $\neg p$, then $\emptyset$ is a rational position, as are $\{p\}$ and $\{\neg p\}$... Which of these positions should we choose? How do we find such a position? This is what argumentation is about.

16.1 Types of Argument

Typically, argumentation involves agents putting forward arguments for and against propositions, together with justifications for the acceptability of these arguments.

The philosopher Michael Gilbert suggests that if we consider argumentation as it occurs between humans, we can identify at least four different modes of argument [Gilbert, [1994](#)] as follows.

Logical mode The logical mode of argumentation resembles mathematical proof. It tends to be deductive in nature (‘if you accept that A and that A implies B , then you must accept that B ’). The logical mode is perhaps the paradigm example of argumentation. It is the kind of argument that we generally expect (or at least hope) to see in scientific papers.

Emotional mode The emotional mode of argumentation occurs when appeals are made to feelings, attitudes, and the like. An example is the ‘how would you feel if it happened to you?’ type of argument.

Visceral mode The visceral mode of argumentation is the physical, social aspect of human argument. It occurs, for example, when one argumentation participant stamps their feet to indicate the strength of their feeling.

Kisceral mode Finally, the kisceral mode of argumentation involves appeals to the intuitive, mystical, or religious.

Of course, depending on the circumstances, we might not be inclined to accept some of these modes of argument. In a court of law in many societies, for example, the emotional and kisceral modes of argumentation are not permitted. Of course, this does not stop lawyers trying to use them: one of the roles of a judge is to rule such arguments unacceptable when they occur. Other societies, in contrast, explicitly allow for appeals to be made to religious beliefs in legal settings. Similarly, while we might not expect to see arguments based on emotion accepted in a court of law, we might be happy to permit them when arguing with our children or spouse.

In this chapter, we will focus on two approaches to automated argumentation. The first approach takes a very abstract view of arguments, not concerned with the actual meaning of arguments, but simply which arguments can ‘coexist’ with each other: this is *abstract argumentation*. The second approach is closely related to logical reasoning, and is called *deductive argumentation*.

ABSTRACT ARGUMENTATION

DEDUCTIVE ARGUMENTATION

16.2 Abstract Argumentation

Suppose you are a juror in a criminal case. You listen to all the evidence put forward by the prosecution and by the defence, and eventually, the jury is asked to retire to consider its verdict. How do you form a judgement? One procedure might be the following. Create a directed graph, with nodes in the graph corresponding to all the arguments put forward. Draw an arc from argument *a* to argument *a'* if accepting argument *a* would mean rejecting argument *a'*. Alternatively, and more commonly, we simply say that *a attacks* argument *a'*. Now, repeat this for all the arguments put forward. What you have constructed by this process is an *abstract argumentation system* (or *Dung-style argumentation system*, after their inventor Phan Minh Dung) [Dung, 1995; Vreeswijk and Prakken, 2000]. In this section, the aim is to show how you can use Dung-style argumentation systems to construct rationally justifiable positions.

ATTACK RELATION

DUNG-STYLE ARGUMENT SYSTEM

Let us consider a small example. Consider the Dung-style argumentation system

$$(\{p, q, r, s\}, \{(r, q), (s, q), (q, p)\}).$$

This argument system is illustrated in [Figure 16.1a](#): vertices are arguments and edges indicate attacks. In this system, the arguments *r* and *s* have no attackers, while *q* is attacked by both *r* and *s*, and *p* is attacked by *q*. [Figures 16.1b](#) and [\(c\)](#) give more complex Dung-style argument

systems.

Given these basic definitions, we can define various notions of when an argument is rationally justifiable. Unfortunately, there is no universally accepted definition of acceptability – there are a number of different notions, each with their own advantages and disadvantages. Here we describe the key ones.

16.2.1 Preferred extensions

The first notion of acceptability that we define is that of a *preferred extension*. First, we say that a *position* is simply a set of arguments. A position S is *conflict free* if no member of S attacks any other member of S . Conflict freeness can be thought of as the most basic requirement for a rational position: it basically says that an argument set is internally consistent. With respect to [Figure 16.1a](#), the following argument sets are conflict free:

PREFERRED EXTENSION

CONFLICT FREE ARGUMENT SET

$$\emptyset, \{p\}, \{q\}, \{r\}, \{s\}, \{r, s\}, \{p, r\}, \{p, s\}, \{r, s, p\}.$$

It may seem strange that we include $\emptyset$ here, but the point is that it satisfies the conditions: there is no attack between any two arguments in $\emptyset$.
If an argument a is attacked by some other argument a' , then we say that a is *defended* if there is some argument a'' that attacks a' . In this case, we say that a is defended by a'' . In [Figure 16.1a](#), argument p is defended by arguments r and s .

DEFENDED ARGUMENT

A position S is *mutually defensive* if every element of S that is attacked is defended by some element of S . We can think of an argument set being mutually defensive if it defends itself against all attacks. It is worth pointing out that we explicitly allow for the possibility of an argument defending itself – the defence does not have to come from another argument. With respect to [Figure 16.1a](#), the following positions are mutually defensive:

MUTUALLY DEFENSIVE ARGUMENT SET

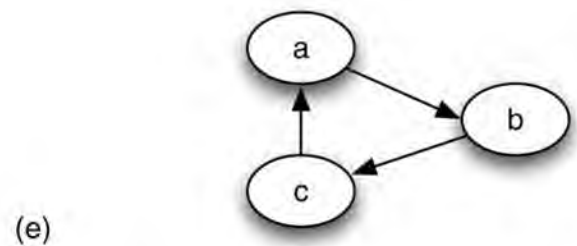
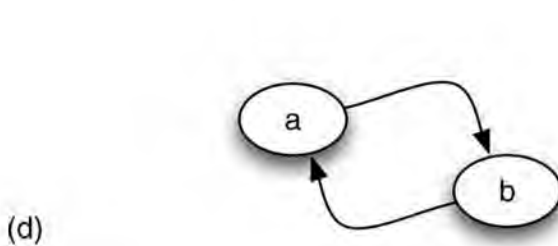
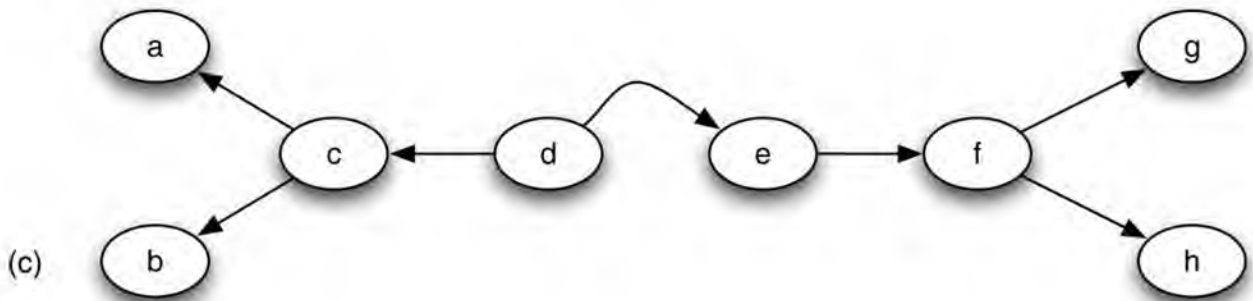
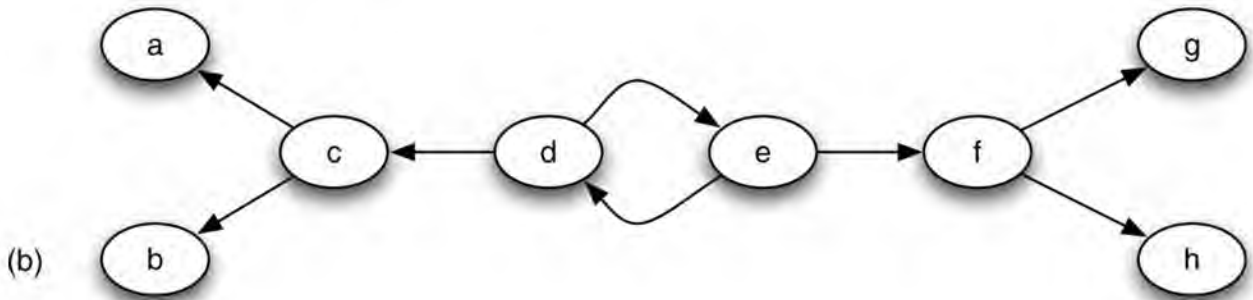
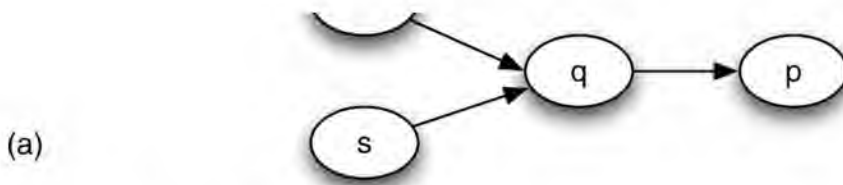
$$\emptyset, \{r\}, \{s\}, \{r, s\}, \{p, r\}, \{p, s\}, \{r, s, p\}.$$

For example, $\{p, r\}$ is mutually defensive because the attack on p by q is defended by r .
Now, we say that a position is *admissible* if it is both conflict free and mutually defensive. The notion of admissibility represents our most basic and most important concept of a rationally justifiable position. An admissible position is one that is internally consistent, and which defends itself against all attackers. With respect to [Figure 16.1a](#), the following positions are admissible:

ADMISSIBLE SET

Figure 16.1: Some example Dung-style abstract argument systems.





$\emptyset, \{r\}, \{s\}, \{r, s\}, \{p, r\}, \{p, s\}, \{r, s, p\}.$

When evaluating argument systems, we usually want to find a position that encompasses as much of the available evidence as possible. In the above example, the fact that $\emptyset$ is an admissible position does not really help us very much. For this reason we define the notion of a *preferred extension*. A preferred extension is an admissible set which has the property that no argument can be added to it without it failing to be admissible: S is admissible but every superset of S is inadmissible. For example, $\emptyset$ is *not* a preferred extension, since $\{p\}$ is a superset of $\emptyset$ and, as we already noted, $\{p\}$ is admissible.

PREFERRED EXTENSION

MAXIMAL ARGUMENT SET

Now, consider [Figure 16.1a](#), above. In this system, the position $\{p, r, s\}$ is conflict free, since no member of this set attacks any other member. However, any set containing q and at least one other member is not conflict free, since both r and s attack q , and q attacks p . The set $\{p, r, s\}$ is admissible, and since it is not possible to add any argument to this set without making it inadmissible, then $\{p, r, s\}$ is a preferred extension. In fact, it is the *only* preferred extension of [Figure 16.1a](#).

Let us consider some slightly more complex examples. Consider [Figure 16.1b](#). One problem with the definitions we have been discussing immediately becomes apparent: a position is a subset of the arguments in the system, and so if there are n arguments, there are 2^n possible positions. Considering each of these positions in turn in order to try to establish whether it is a preferred extension is not feasible for any but the most trivial argument system. We return to this point below. Now, in this system, there will be just two preferred extensions:

$$\{a,b,d,f\} \text{ and } \{c,e,g,h\}.$$

Before proceeding, convince yourself of this!

Now consider [Figure 16.1c](#). This system is almost the same as [Figure 16.1b](#), the only difference being that the edge from e to d is no longer present. What preferred extensions does this system have? In fact, it has only one: $\{a, b, d, f\}$. Arguments c and e are now attacked but undefended, and so cannot be in any admissible set. Since g and h are attacked, and e is the only candidate to be a defender of g and h , this means that g and h cannot be in an admissible set either.

[Figures 16.1d](#) and [\(e\)](#) illustrate some of the limitations of preferred extensions. The argument system in [Figure 16.1d](#) has two preferred extensions ($\{a\}$ and $\{b\}$), while the only preferred extension of [Figure 16.1e](#) is $\emptyset$. In neither case does the preferred extension seem to help us to understand what is going on. Nevertheless, the concept of a preferred extension provides perhaps the most common and important definition of when an argument is ‘reasonable’. [Dung, [1995](#)] proved that every Dung-style argument system has at least one preferred extension, although, as in [Figure 16.1e](#), this may be the empty set.

If we are going to be computing with notions such as admissible sets and preferred extensions, then it is important to know how complex such solutions are. This question was addressed in [Dimopolous and Torres, [1996](#); Dunne, [2007](#); Dunne and Bench-Capon, [2002](#)]. Admissible sets are easy to deal with, computationally: checking whether a given set S of arguments is admissible can be done in polynomial time. Preferred extensions, however, are computationally harder: checking whether a given set S is a preferred extension is co-NP-complete.

16.2.2 Credulous and sceptical acceptance

While preferred extensions are one of the most important solutions in abstract argument systems, as we noted above there are a number of difficulties associated with them, chief among them being that an argument system can have multiple preferred extensions. In this case, how are we to choose between them? Suppose we are trying to determine the status of two arguments, a and b . One obvious way to address this is to consider *how many times* a and b occur in the different preferred extensions. If a occurs in more preferred extensions than b , for example, then we can take this as evidence that a is in some sense ‘stronger’ than b .

Figure 16.2: Computing a grounded extension.

```
function ge(X,A) returns a subset of X
1.  in ← out ← ∅
2.  while in ≠ X do
3.      in ← {α ∈ X; ∄α' ∈ X s.t. (α', α) ∈ A}
4.      out ← {α ∈ X; ∃α' ∈ in s.t. (α', α) ∈ A}
5.      X ← X \ out
6.  return in
```



```

6.       $A \leftarrow A$  restricted to  $\Lambda$ 
7.  end-while
8.  return  $X$ .
9. end function  $ge$ 

```

We say that a is *sceptically accepted* if it is a member of *every* preferred extension, and *credulously accepted* if it is a member of at least one preferred extension.¹ Since there will always be at least one preferred extension, it follows that any argument that is sceptically accepted is also credulously accepted.

SCEPTICAL ACCEPTANCE

CREDULOUS ACCEPTANCE

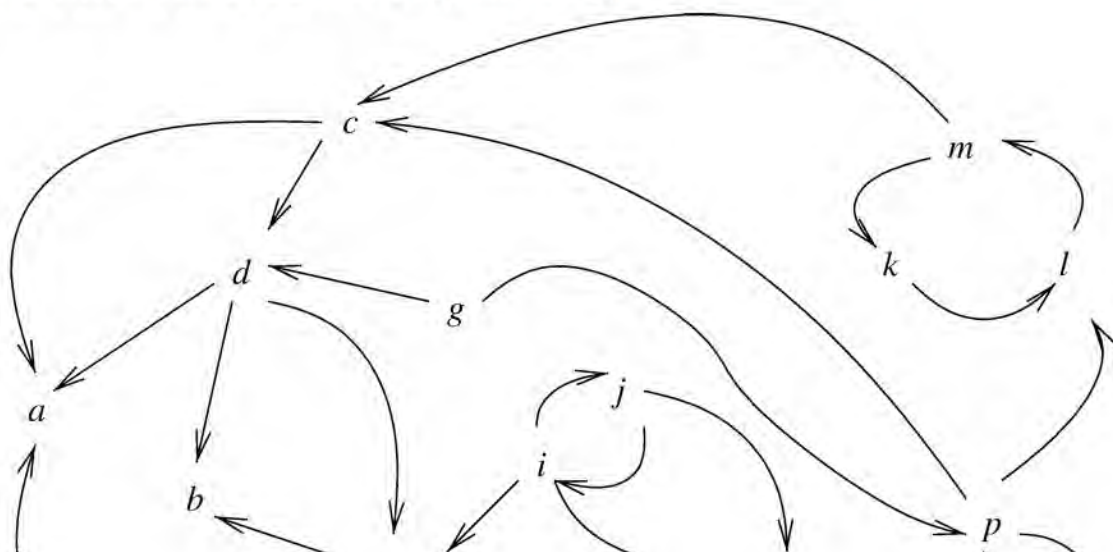
Returning again to [Figure 16.1a](#), the arguments p , r , and s are all sceptically accepted (and hence credulously accepted). The argument q is neither sceptically nor credulously accepted.

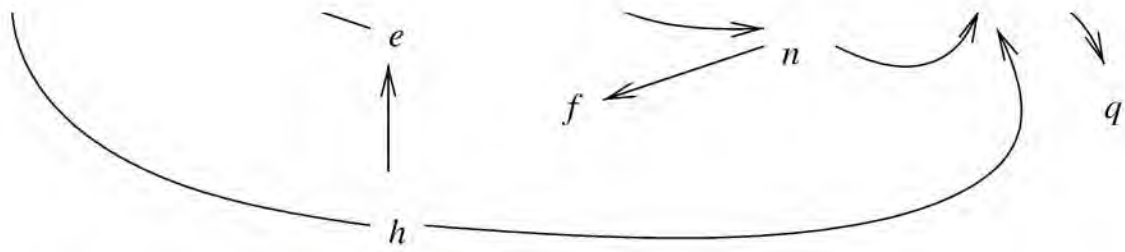
Grounded extensions

An alternative notion of an acceptable set of arguments, also defined by Dung, is that of the *grounded extension*. The idea is as follows. Given an abstract argument system, we start by looking for arguments that are *guaranteed* to be acceptable: those that could not possibly be unacceptable under any sensible definition of acceptability. The most obvious candidates for such acceptable arguments are those that have *no attackers whatsoever*: for surely, the status of arguments that have no attackers whatsoever must be beyond question. Having identified such arguments, we can then proceed to eliminate those arguments that are attacked by these: since these arguments are attacked by arguments that are surely ‘in’, then these arguments must surely be ‘out’. We can then iterate this process, deleting arguments that are attacked by arguments that are surely ‘in’, until we find no changes occurring to the argument graph that remains. We refer to the arguments in the graph that remains after this process has terminated as the grounded extension of the argument system. The algorithm to compute the grounded extension of a Dung argument system is given in [Figure 16.2](#): here, X is the initial set of arguments, and A is the initial attack relation.

GROUNDED EXTENSION

Figure 16.3: An abstract argument system.





If we apply the algorithm in [Figure 16.2](#) to the argument system [Figure 16.1a](#), then we obtain $\{r, s, p\}$ as the grounded extension.

The main advantage of the notion of grounded extension is that there will always be such an extension, and this extension is guaranteed to be unique. However, if there are no arguments that are free of attackers initially, then the grounded extension will be empty.

[Figure 16.3](#) (from [Vreeswijk and Prakken, 2000]) illustrates an abstract argument system. With respect to this example:

- argument h has no attackers, and so is clearly acceptable ('in')
- since h is in, and h attacks a , then a is not an acceptable argument – it is out
- similarly, since h is in, and h attacks p , then p is out
- since p is out, and this is the only attacker of q , then q is in
- since h is in, and h attacks e , then e is out
- ... and so on.

16.2.3 Preferences in abstract argument systems

It could be argued that one reason why the standard model of abstract argument systems does not fare well in some cases when attempting to distinguish the status of arguments is that 'all arguments are equal', in the sense that all attacks are equally strong. In reality, of course, nothing could be further from the truth: some arguments are stronger than others. For example, in a court of law, we would typically be inclined to give more credence to arguments formed on the basis of forensic evidence rather than arguments formed on the basis of hearsay. An obvious avenue for developing models of argument is therefore to try to capture notions of relative strength, which leads to *preference-based argumentation* models [Amgoud, 1999; Amgoud and Cayrol, 1998].

PREFERENCE-BASED ARGUMENTATION

In a preference-based argumentation model, arguments are *stratified*: that is, divided up into a hierarchy of tiers, with arguments in lower tiers being said to be *more preferred* than those in higher tiers.² We write $a \succeq b$ to indicate that argument a is at least as low in the argument hierarchy as b , and $a \triangleleft b$ to indicate that a is lower in the hierarchy than b (remember, the lower in the hierarchy an argument is, the stronger the argument is).

STRATIFIED ARGUMENT SET

We can then refine the standard notion of attack a little: an argument a is said to attack b if there is an edge from a to b in the attack graph, and moreover, it is not the case that $b \triangleleft a$. Notice that an argument can never attack an argument in a lower tier. Given this notion of attack, we can define the acceptable arguments. We start with the set C of arguments that

defend themselves: an argument a defends itself if, for every attack on this argument by b , either a is preferred over b ($a \triangleleft b$) or else a attacks b in return. If S is a set of arguments, then denote by $F(S)$ the set of arguments that S defends. Thus $F(\emptyset) = C$. We then define the *acceptable* arguments to be:

$$C \cup F(C) \cup F(F(C)) \cup F(F(F(C))) \cup \dots$$

In other words, we start by finding the arguments that defend themselves; then we find the arguments that these arguments defend; then find the arguments that these arguments defend, and so on. It may seem that this is an infinite construction, but in fact, since we only have a finite number of arguments, and since the set of acceptable arguments grows monotonically with every application of $F(\dots)$, we will only have to iterate the application of F a finite number of times before we find that we have no change in the set of acceptable arguments, and we can stop. (Technically, we are computing a *least fixed point* of $F(\dots)$.)

16.2.4 Values in abstract argument systems

Another approach to distinguishing between different argument systems involves considering the *values* that an argument appeals to, and how different audiences *rank* these values [Bench-Capon, 2003; Bench-Capon and Sartor, 2003; Bench-Capon et al., 2007]. To see what I mean by this, consider the following scenario:

VALUE-BASED ARGUMENTATION

Alice lives peacefully in her house until one night a burglar, Bob, breaks in. Alice kills Bob.

Should Alice be punished? We might expect to hear something like the following arguments presented with respect to this scenario:

- Alice has the right to protect her property from burglars; therefore Alice should not be punished.
- Everybody has the right to life; therefore Alice should be punished.

Unless you are either exceedingly liberal or exceedingly hawkish, I would expect that you can see that both arguments have *some* merit. In a logical sense, they are both sound. But they *appeal to different values*, and how we choose between them depends largely on how we rank these values. The values in question are the right to protect one's property, and the right to life. If we rank the right to protect one's property above the right to life, then one would be inclined to accept the first argument rather than the second. On the other hand, if you rank the right to life above the right to protect one's property, then you would probably be inclined to accept the second argument rather than the first. In many countries (including the UK), courts seem to rank the right to life above the right to protect property. In this situation, as Alice's lawyer, you might be inclined to introduce an argument appealing to another value: the right to protect one's own life:

- Alice believed Bob was about to kill her, and acted in self defence; therefore Alice should not be punished.

I would hazard a guess that most people would rank the right of Alice to protect her own life above the right to life of someone attacking her. If Alice's lawyer could successfully argue that Alice believed her life to be at imminent risk from Bob, she would probably go

unpunished.

Incorporating values into Dung-style abstract argument frameworks is technically simple. We have our set of arguments X as before, with the associated attack relation, but then for every argument a we have a *value*, $\eta(a)$. In addition, we have *audiences*, and for every audience A we have a ranking relation $\triangleright_A$ on values, where $v_1 \triangleright_A v_2$ means that audience A regards value v_1 as being more important than v_2 . If argument a attacks another argument b , then we say that the argument succeeds with respect to audience A if it is not the case that $\eta(b) \triangleright_A \eta(a)$. Notice that the notion of attack is now *relativized* to a particular audience.

Given this idea, we can talk about arguments being acceptable for a particular audience (this is called *subjective acceptance*), being acceptable for *every* audience (*objective acceptance*), or being acceptable for *no* audience (in which case we say the argument is *indefensible*).

SUBJECTIVE ACCEPTANCE

Computationally, the problems of checking subjective and objective acceptance are both hard (NP-complete and co-NP-complete respectively).

OBJECTIVE ACCEPTANCE

INDEFENSIBLE ARGUMENT

16.3 Deductive Argumentation Systems

One feature of abstract argumentation systems is that the arguments themselves have no *structure* – an argument is assumed to be an indivisible, atomic thing. In practice, however, arguments may well be complex beasts, intricately built up from many components with a logical structure. In some settings, it is helpful to recognize this structure. In particular, as we will see, this can give a precise meaning to the notion of ‘attack’ between arguments. In this subsection, I introduce *deductive argumentation* [Besnard and Hunter, 2008]. In deductive argumentation, arguments are built from logical formulae, and the notion of ‘attack’ between arguments is built from the idea of logical proof.

DEDUCTIVE ARGUMENTATION

In classical logic, an argument is a sequence of inferences leading to a conclusion: we write $\Delta \vdash \phi$ to mean that there is a sequence of inferences from premises Δ that will allow us to establish conclusion ϕ . Consider the simple database Δ_1 which expresses some very familiar information in a Prolog-like notation in which variables are capitalized and ground terms and predicate names start with small letters:

$$\begin{aligned} &human(Socrates). \quad \Delta_1 \\ &human(X) \rightarrow mortal(X). \end{aligned}$$

The claim $\Delta_1 \vdash mortal(Socrates)$ may be correctly made from this database because $mortal(Socrates)$ follows from Δ_1 given the usual logical axioms and rules of inference of classical logic. Thus a correct argument simply yields a conclusion which in this case could be paraphrased ‘ $mortal(Socrates)$ is true in the context of $human(Socrates)$ and $human(X) \rightarrow mortal(X)$ ’. In the system of argumentation we adopt here, this traditional form of reasoning is

mortal(A). In the system of argumentation we adopt here, this traditional form of reasoning is extended by explicitly recording those propositions that are used in the derivation. This makes it possible to assess the basis of a given argument by examining the propositions upon which it depends.

A bit more formally, we will assume that we have a set of logical formulae, DB , which represent the ‘evidence’ available to us. As we noted above, it may be that DB is not consistent. Given DB , a *deductive argument* is a pair:

DEDUCTIVE ARGUMENT

(*Support, Conclusion*)

such that:

1. $Support \subseteq DB$ (the argument is based on the available evidence)
2. $Support \vdash Conclusion$ (the conclusion can be proved from *Support*)
3. *Support* is logically consistent
4. there is no subset of *Support* satisfying the above conditions.

The intuition is that DB is a set of formulae that is ‘agreed’ between the agents participating in the argumentation process. This database provides some common ground between the agents. Given this common ground, an agent makes the argument (*Support, Conclusion*) in support of the claim that *Conclusion* is true; the justification for this claim is provided by *Support*, which is a set of formulae such that *Conclusion* can be proved from it.

We denote the set of all such arguments over database DB by $A(DB)$, and use $Arg, Arg', Arg_1, \dots$ to stand for members of $A(DB)$.

We can now consider the notion of attack (or, to put it from another perspective, defeat) between arguments.

Attack For any two propositions ϕ and ψ , we say that ϕ attacks ψ if and only if $\phi \models \neg \psi$.

Defeat Let (Γ_1, ϕ_1) and (Γ_2, ϕ_2) be arguments from some database DB . The argument (Γ_2, ϕ_2) can be defeated in one of two ways.

- (Γ_1, ϕ_1) *rebut*s (Γ_2, ϕ_2) if ϕ_1 attacks ϕ_2 .
- (Γ_1, ϕ_1) *undercuts* (Γ_2, ϕ_2) if ϕ_1 attacks ψ for some $\psi \in \Gamma_2$.

REBUTTAL

UNDERCUT

Consider the following set of formulae, which extend the example of Δ_1 with information in common currency at the time of Plato:

human(Heracles)
father(Heracles, Zeus)
father(Apollo, Zeus)

$$\begin{aligned}
& \text{divine}(X) \rightarrow \neg \text{mortal}(X) \\
& \text{father}(X, \text{Zeus}) \rightarrow \text{divine}(X) \\
& \neg(\text{father}(X, \text{Zeus}) \rightarrow \text{divine}(X)).
\end{aligned}$$

From this we can build the obvious argument, Arg_1 , about *Heracles*,

$$(\{\text{human}(\text{Heracles}), \text{human}(X) \rightarrow \text{mortal}(X)\}, \text{mortal}(\text{Heracles}))$$

as well as a rebutting argument Arg_2 ,

$$(\{\text{father}(\text{Heracles}, \text{Zeus}), \text{father}(X, \text{Zeus}) \rightarrow \text{divine}(X), \text{divine}(X) \rightarrow \neg \text{mortal}(X)\}, \neg \text{mortal}(\text{Heracles})).$$

The second of these arguments is undercut by Arg_3 :

$$(\{\neg(\text{father}(X, \text{Zeus}) \rightarrow \text{divine}(X))\}, \neg(\text{father}(X, \text{Zeus}) \rightarrow \text{divine}(X)))$$

We can define an ordering over argument types, which tries to capture the notion of increasing acceptability. The idea is that, when engaged in argumentation, we intuitively recognize that some types of argument are more ‘powerful’ than others. For example, given database $DB = \{p \rightarrow q, p\}$, the arguments $\text{Arg}_1 = (\emptyset, p \vee \neg p)$ and $\text{Arg}_2 = (\{p \rightarrow q, p\}, q)$ are both acceptable members of $A(DB)$. However, it is generally accepted that Arg_1 – a tautological argument – is stronger than Arg_2 , for the simple reason that it is not possible to construct a scenario in which the conclusion of Arg_1 is false. Any agent that accepted classical propositional logic would have to accept Arg_1 (but an agent that did not accept the ‘law of the excluded middle’ – that $p \vee \neg p$ is a tautology – would not). In contrast, the argument for the conclusion of Arg_2 depends on two other propositions, both of which could be questioned.

In fact, we can identify five classes of argument type, which we refer to as A_1 to A_5 , respectively. In order of increasing acceptability, these are as follows.

A_1 The class of all arguments that may be made from DB .

A_2 The class of all non-trivial arguments that may be made from DB .

A_3 The class of all arguments that may be made from DB for which there are no rebutting arguments.

A_4 The class of all arguments that may be made from DB for which there are no undercutting arguments.

A_5 The class of all tautological arguments that may be made from DB .

There is an order, $\preccurlyeq$, over the acceptability classes:

$$A_1(DB) \preccurlyeq A_2(DB) \preccurlyeq A_3(DB) \preccurlyeq A_4(DB) \preccurlyeq A_5(DB),$$

meaning that arguments in higher numbered classes are *more acceptable* than arguments in lower numbered classes. The intuition is that there is less reason for thinking that there is something wrong with them – because, for instance, there is no argument which rebuts them. The idea that an undercut attack is less damaging than a rebutting attack is based on the notion that an undercut allows for another, undefeated, supporting argument for the same conclusion. This is common in the argumentation literature [Krause et al., 1995].

In the previous example, the argument

$$(\emptyset, \text{divine}(\text{Heracles}) \vee \neg \text{divine}(\text{Heracles}))$$

is in A_5 , while Arg_1 and Arg_2 are mutually rebutting and thus in A_2 , whereas Arg_4 ,

$$(\{\text{father}(\text{Apollo}, \text{Zeus}), \text{father}(X, \text{Zeus}) \rightarrow \text{divine}(X), \text{divine}(X) \rightarrow \neg \text{mortal}(X)\}, \neg \text{mortal}(\text{Apollo}))$$

is in A_4 .

This logic-based model of argumentation has been used in argumentation-based negotiation systems [Parsons and Jennings, 1996; Parsons et al., 1998]. The basic idea is as follows. You are attempting to negotiate with a peer over who will carry out a particular task. Then the idea is to argue for the other agent intending to carry this out, i.e. you attempt to convince the other agent of the acceptability of the argument that it should intend to carry out the task for you.

16.4 Dialogue Systems

Many authors are concerned with agents that argue with themselves, either to resolve inconsistencies or else to determine which set of assumptions to adopt. In contrast, we are interested in agents that are involved in a *dialogue* with other agents. As we noted above, an agent engages in such a dialogue in order to convince another agent of some state of affairs. In this section, we define the notion of dialogue, and investigate the concept of *winning* an argument. Call the two agents involved in argumentation 0 and 1.

DIALOGUE

Intuitively, a dialogue is a series of arguments, with the first made by agent 0, the second by agent 1, the third by agent 0, and so on. Agent 0 engages in the dialogue in order to convince agent 1 of the conclusion of the first argument made. Agent 1 attempts to defeat this argument, by either undercutting or rebutting it. Agent 0 must respond to the counter argument if it can, by presenting an argument that defeats it, and so on. (For a concrete example of how this kind of argumentation can be used to solve negotiation problems, see [Parsons et al., 1998] and [Amgoud, 1999].)

Each step of a dialogue is referred to as a *move*. A move is simply a pair $(Player, Arg)$, where $Player \in \{0,1\}$ is the agent making the argument, and $Arg \in A(DB)$ is the argument being made. I use m (with decorations: $m, m', \dots$ and so on) to stand for moves.

Formally, a non-empty, finite sequence of moves

$$(m_0, m_1, \dots, m_k)$$

is a *dialogue history* if it satisfies the following conditions.

1. $Player_0 = 0$ (the first move is made by agent 0).
2. $Player_u = 0$ if and only if u is even, $Player_u = 1$ if and only if u is odd (the agents take it in turns to make proposals).
3. If $Player_u = Player_v$ and $u \neq v$ then $Arg_u \neq Arg_v$ (agents are not allowed to make the same argument twice).
4. Arg_σ defeats Arg_τ .

Consider the following dialogue in which agent 0 starts by making Arg_1 for r :

$$m_0 = (\{p, p \rightarrow q, q \rightarrow r\}, r).$$

Agent 1 undercuts this with an attack on the connection between p and q ,

$$m_1 = (\{t, t \rightarrow \neg(p \rightarrow q)\}, \neg(p \rightarrow q)),$$

and agent 0 counters with an attack on the premise t using Arg_3 ,

$$m_2 = (\{s, s \rightarrow \neg t\}, \neg t).$$

A dialogue has *ended* if there are no further moves possible. The *winner* of a dialogue that has ended is the last agent to move. If agent 0 was the last agent to move, then this means that agent 1 had no argument available to defeat 0’s last argument. If agent 1 was the last agent to move, then agent 0 had no argument available to defeat 1’s last argument. Viewed in this way, argument dialogues can be seen as a game played between proposers and opponents of arguments.

[Walton and Krabbe, 1995, p. 66] suggest a typology of seven different modes of dialogues, which are summarized in Table 16.1. The first (type I) involves the ‘canonical’ form of argumentation, where one agent attempts to convince another of the truth of something. Initially, agents involved in persuasion dialogues will have conflicting opinions about some state of affairs. To use a classic, if slightly morbid, example, you may believe the murderer is Alice, while I believe the murderer is Bob. We engage in a persuasion dialogue in an attempt to convince one another of the truth of our positions.

Table 16.1: Walton and Krabbe’s dialogue types.

	Type	Initial situation	Main goal	Participant’s aim
I.	Persuasion	conflict of opinions	Resolve the issue	Persuade the other
II.	Negotiation	Conflict of interests	Make a deal	Get the best for oneself
III.	Inquiry	General ignorance	Growth of knowledge	Find a ‘proof’
IV.	Deliberation	Need for action	Reach a decision	Influence outcome
V.	Information seeking	Personal ignorance	Spread knowledge	Gain or pass on personal knowledge
VI.	Eristics	Conflict/antagonism	Reaching an accommodation	Strike the other party
VII.	Mixed	Various	Various	Various

In a persuasion dialogue, the elements at stake are primarily beliefs. In contrast, a negotiation (type II) dialogue directly involves utility. It may involve attempting to reach agreement on a division of labour between us.

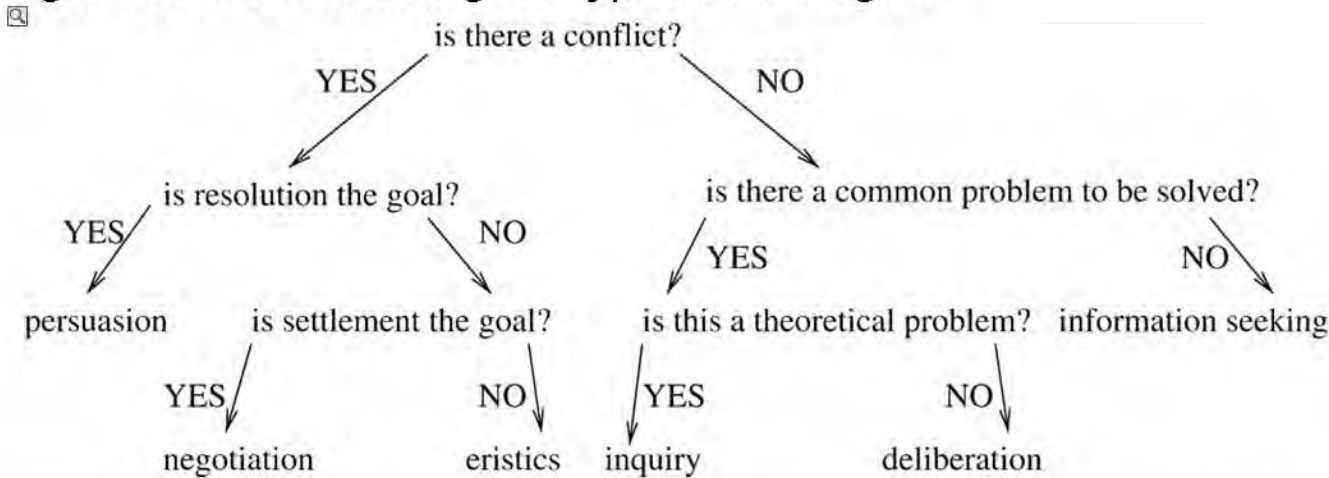
An inquiry (type III) dialogue is one that is related to a matter of common interest, where the object of the inquiry is a belief. A public inquest into some event (such as a train crash) is perhaps the best-known example of an inquiry. It takes place when a group of people have

perhaps the best known example of an inquiry. It takes place when a group of people have some mutual interest in determining something. Notice that the aim of an inquiry is simply to determine facts – what to believe. If the aim of a dialogue is for a group to decide upon a course of action, then the dialogue is a deliberation dialogue. An information-seeking (type V) dialogue is also related to an inquiry, but occurs when an agent attempts to find out something for itself. An eristic (type VI) dialogue occurs when agents have a conflict that they air in public. The aim of such a dialogue may be to reach an accommodation, but need not be. Finally, type VII or mixed dialogues occur when a number of different dialogue types are combined. Most committee meetings are of this kind: different parts of the meeting involve negotiation, deliberation, inquiry, and, frequently, eristic dialogues. [Figure 16.4](#) shows how the type of a dialogue may be determined [Walton and Krabbe, [1995](#), p. 81].

16.5 Implemented Argumentation Systems

Several agent systems have been developed which make use of argumentation-based negotiation. Probably the first of these was Sycara’s Persuader system [Sycara, [1989a](#), [b](#), [1990](#)]. Persuader operated in the domain of labour negotiation, and involved three agents (a labour union, a company, and a mediator). It modelled the iterative exchange of proposals and counter-proposals in order for the parties to reach agreement. The negotiation involved multiple issues (such as wages, pensions, seniority, and subcontracting).

Figure 16.4: Determining the type of a dialogue.



Argumentation in Persuader makes use of a model of each agent’s beliefs. An agent’s beliefs in Persuader capture an agent’s goals and the interrelationships among them. An example of an agent’s beliefs (from [Sycara, [1989b](#)]) is given in [Figure 16.5](#). This captures the beliefs of a company, the top-level goal of which is to maximize profit. So, for example, a decrease (–) in production costs will lead to an increase (+) in profit; an increase in quality or a decrease in prices will lead to an increase in sales, and so on. [Sycara, [1989b](#)] gives an example of the following argument, addressed to a labour union that has refused a proposed wage increase:

If the company is forced to grant higher wage increases, then it will decrease employment.

To generate this argument, the system determines which goals (illustrated in [Figure 16.5](#)) are violated by the union’s refusal, and then looks for compensating actions. In this case, a compensating action might be to reduce employment, either by subcontracting or increasing automation. Such a compensating action can violate a goal that the union rates more highly than higher wages. [Figure 16.6](#) illustrates a run of Persuader (from [Sycara, [1989b](#)]), showing how the system generates the argument from the belief structure in [Figure 16.5](#).

In general, Persuader can generate more than one possible argument for a particular position. These arguments are presented in order of 'severity', with the weakest type of argument first. The order of argument types (weakest first) is as follows [Sycara, 1989b, p. 131]:

- 1. appeal to universal principle

Figure 16.5: Argument structure in the PERSUADER system.

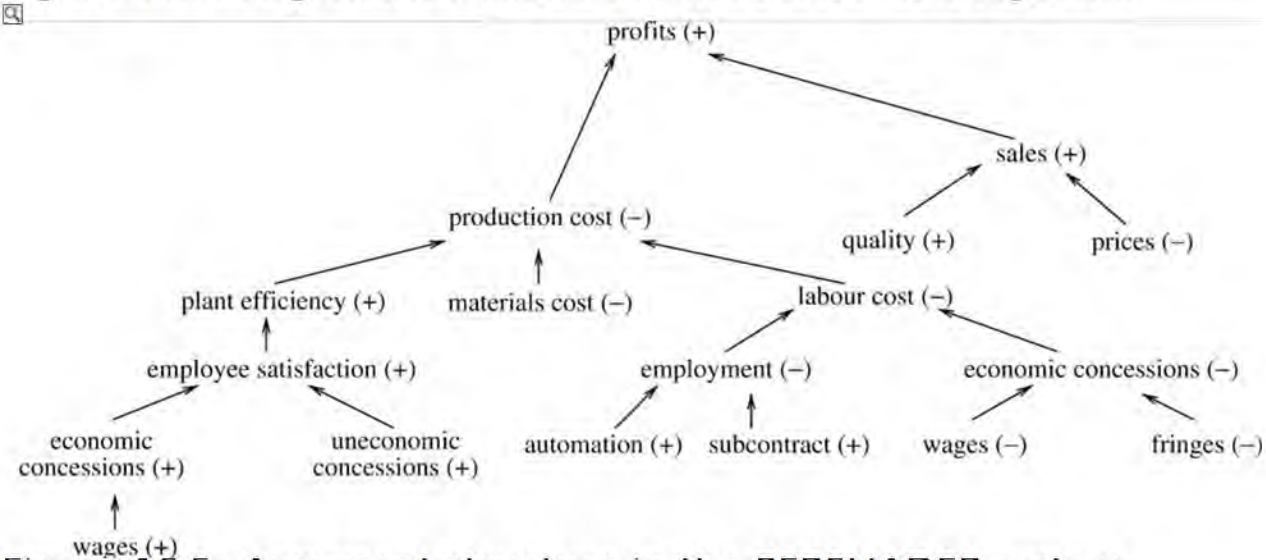


Figure 16.5: Argument structure in the PERSUADER system.

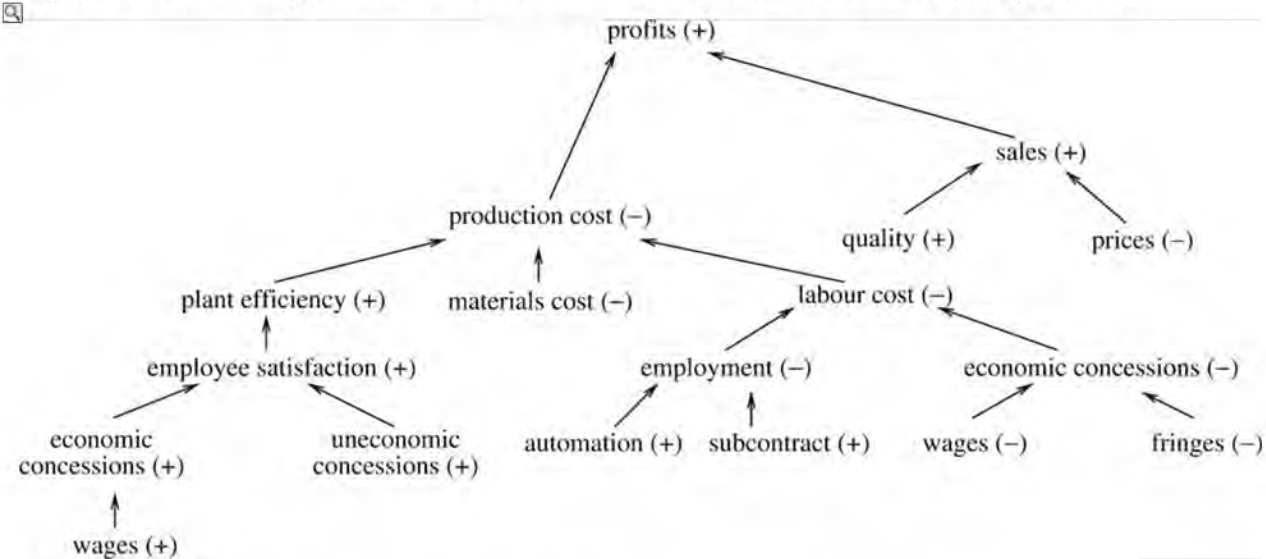


Figure 16.6: Persuader generates an argument.

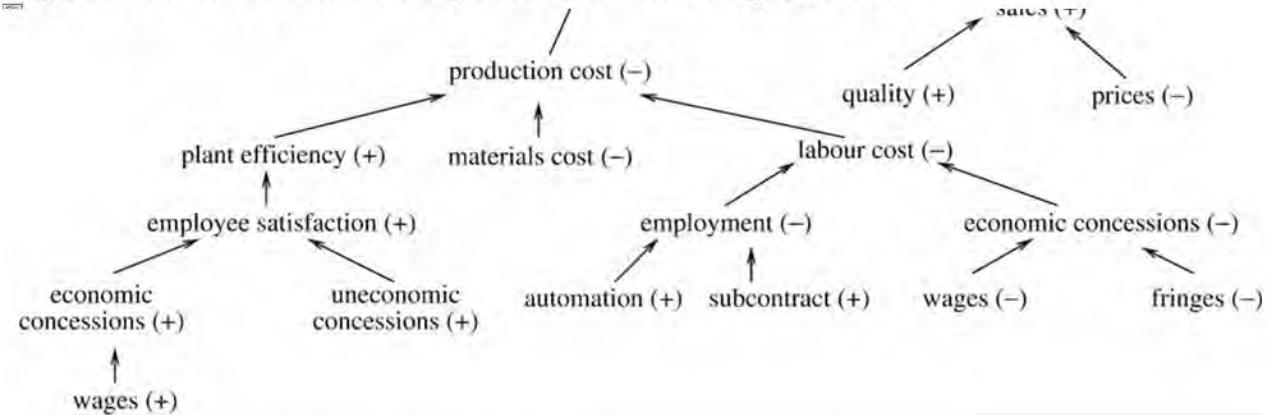


Figure 16.6: Persuader generates an argument.

Importance of wage-goal1 is 6 for union1
Searching company1 goal-graph...
Increase in wage-goal1 by companv1 will result in

```

    increase in economic-concessions, labour-cost1, production-cost1
Increase in wage-goall by company1 will result in
    decrease in profits1
To compensate, company1 can decrease fringe-benefits1,
    decrease employment1, increase plant-efficiency1,
    increase sales1
Only decrease fringe-benefits1, decreases employment1
    violate goals of union1
Importance of fringe-benefits1 is 4 for union1
Importance of employment1 is 8 for union1

```

Figure 16.6: Persuader generates an argument.

```

Importance of wage-goall is 6 for union1
Searching company1 goal-graph...
Increase in wage-goall by company1 will result in
    increase in economic-concessions, labour-cost1, production-cost1
Increase in wage-goall by company1 will result in
    decrease in profits1
To compensate, company1 can decrease fringe-benefits1,
    decrease employment1, increase plant-efficiency1,
    increase sales1
Only decrease fringe-benefits1, decreases employment1
    violate goals of union1
Importance of fringe-benefits1 is 4 for union1
Importance of employment1 is 8 for union1
Since importance of employment1 > importance of wage-goall
One possible argument found

```

2. appeal to a theme

3. appeal to authority

```

    increase in economic-concessions, labour-cost1, production-cost1
Increase in wage-goall by company1 will result in
    decrease in profits1
To compensate, company1 can decrease fringe-benefits1,
    decrease employment1, increase plant-efficiency1,
    increase sales1
Only decrease fringe-benefits1, decreases employment1
    violate goals of union1
Importance of fringe-benefits1 is 4 for union1
Importance of employment1 is 8 for union1
Since importance of employment1 > importance of wage-goall
One possible argument found

```

2. appeal to a theme

3. appeal to authority

4. appeal to 'status quo'

5. appeal to 'minor standards'

6. appeal to 'prevailing practice'

7. appeal to precedents as counter-examples

8. threaten.

The idea is closely related to the way in which humans use arguments of different 'strength' in

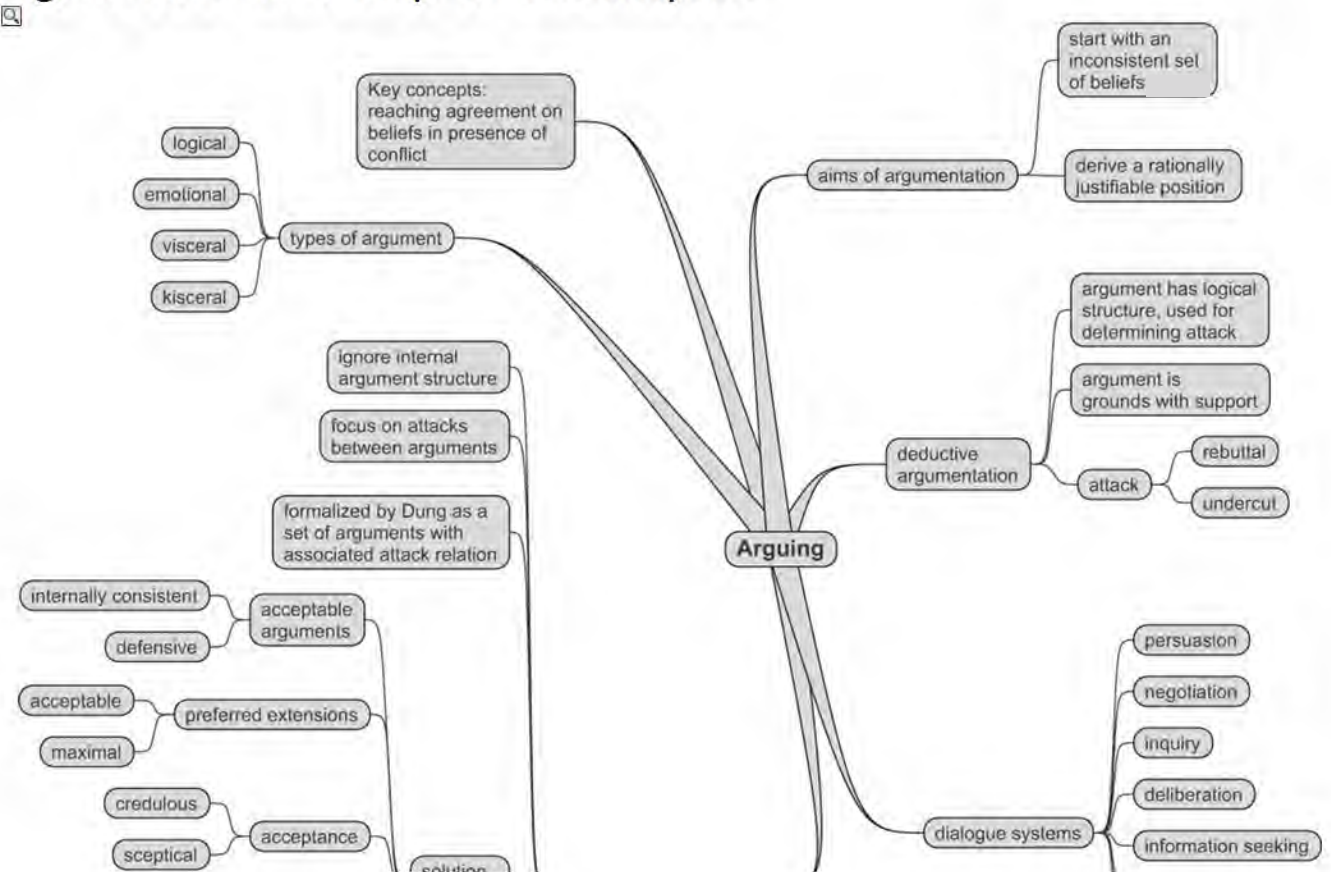
Notes and Further Reading

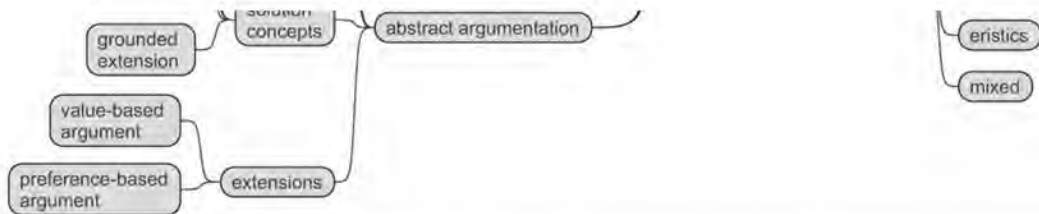
Argumentation was originally studied by philosophers and logicians in an attempt to understand the ‘informal logic’ that humans use to interact with one another [van Eemeren et al., 1996; Walton and Krabbe, 1995]. More recently, argumentation has been found to have a number of applications in AI, particularly in decision-making [Fox et al., 1992; Krause et al., 1995], the semantics of logic programming [Dimpoulos et al., 1999; Dung, 1995], and defeasible reasoning [Loui, 1987; Pollock, 1992, 1994]. A survey of work on argumentation up to 2001 was published as [Prakken and Vreeswijk, 2001], although this does not deal with the subject from the standpoint of multiagent systems. More recently, an excellent advanced introduction to argumentation from the AI perspective was published [Besnard and Hunter, 2008], which represents the most comprehensive overview of the field published to date; this is recommended as essential reading for anyone aiming to gain a deeper understanding of the concepts and techniques in argumentation theory. A thorough annotated bibliography of argumentation from the perspective of AI was published as [Bench-Capon and Dunne, 2007]. A special issue of *IEEE Intelligent Systems* on argumentation was published in November/December 2007 (volume 22, number 6).

An attempt to formalize some of the ideas in Persuader using logic and then to implement this formal version was [Kraus et al., 1998]. A number of authors have proposed the use of variations of Walton and Krabbe’s dialogue types for multiagent systems [Amgoud, 1999; Amgoud et al., 2000; Reed, 1998].

Class reading: [Dung, 1995]. This is the article that transformed the study of argumentation from a philosophical discipline to a scientific one. A huge range of ideas are in evidence here, and the basic definitions are still in common use today.

Figure 16.7: Mind map for this chapter.





¹ Someone is sceptical if they are hard to convince of things, and credulous if they tend to believe things easily

² Beware of terminology here: although we are using the term 'preference', the meaning is not intended to be the same as the meaning the term has elsewhere in this book.

Chapter 17 Logical Foundations

Computer science is, as much as it is about anything, about developing formal theories to specify and reason about computer systems. Many formalisms have been developed in mainstream computer science to do this, and it comes as no surprise to discover that the agents community has also developed many such formalisms. In this chapter, we look at some of the logics that have been developed for reasoning about multiagent systems, and investigate how these logics might be used to prove properties of systems.

We start by looking at the use of logics to represent and reason about the *information* that agents possess: their knowledge and belief. The predominant approach to this problem has been to use what are called *modal logics* to do this. Following an introduction to the need for modal logics for reasoning about agents, we introduce the paradigm of *normal modal logics with Kripke semantics*, as this approach is almost universally used. We then go on to consider *integrated theories of agency*, where we reason not just about knowledge and belief, but also about other mental states – desires and the like. We then consider logics intended to support reasoning about cooperation and strategy, and show how such logics can be used to capture interesting properties of social choice mechanisms. We conclude by discussing the way that these formalisms might actually be used in the development of agent systems.

MODAL LOGIC

Please note: this chapter presupposes some understanding of the use of logic and formal methods for specification and verification. It is probably best avoided by those without such a background.

17.1 Logics for Knowledge and Belief

Suppose one wishes to reason about mental states – beliefs and the like – in a logical framework. Consider the following statement (after [Genesereth and Nilsson, 1987, pp. 210–211]):

$$\text{Janine believes Cronos is the father of Zeus .} \tag{17.1}$$

The best-known and most widely used logic in computer science is first-order logic. So, can we represent this statement in first-order logic? A naive attempt to translate (17.1) into first-order logic might result in the following:

$$\text{Bel (Janine, Father (Zeus , Cronos)).} \tag{17.2}$$

Unfortunately, this naive translation does not work, for at least two reasons. The first is syntactic: the second argument to the *Bel* predicate is a *formula* of first-order logic, and is not, therefore, a term. So (17.2) is not a well-formed formula of classical first-order logic.

The second problem is semantic. The constants *Zeus* and *Jupiter*, by any reasonable interpretation, denote the same individual: the supreme deity of the classical world. It is therefore acceptable to write, in first-order logic,

$$\text{(Zeus = Jupiter).} \tag{17.3}$$

Given (17.2) and (17.3) the standard rules of first-order logic would allow the derivation of the

Given (17.2) and (17.3), the standard rules of first-order logic would allow the derivation of the following:

$$\text{Bel} \left(\text{Janine}, \text{Father} \left(\text{Jupiter}, \text{Cronos} \right) \right). \tag{17.4}$$

But intuition rejects this derivation as invalid: believing that the father of Zeus is Cronos is *not* the same as believing that the father of Jupiter is Cronos.

So what is the problem? Why does first-order logic fail here? The problem is that the intentional notions – such as belief and desire – are *referentially opaque*, in that they set up *opaque contexts*, in which the standard substitution rules of first-order logic do not apply. In classical (propositional or first-order) logic, the denotation, or semantic value, of an expression is dependent solely on the denotations of its subexpressions. For example, the denotation of the propositional logic formula $p \wedge q$ is a function of the truth-values of p and q . The operators of classical logic are thus said to be *truth functional*.

OPAQUE CONTEXT

TRUTH-FUNCTIONAL OPERATORS

In contrast, intentional notions such as belief are *not* truth functional. It is surely not the case that the truth value of the sentence:

$$\text{Janine believes } p \tag{17.5}$$

is dependent solely on the truth-value of p .¹ So substituting equivalents into opaque contexts is not going to preserve meaning. This is what is meant by referential opacity. The existence of referentially opaque contexts has been known since the time of Frege. He suggested a distinction between *sense* and *reference*. In ordinary formulae, the ‘reference’ of a term/formula (i.e. its denotation) is needed, whereas in opaque contexts, the ‘sense’ of a formula is needed (see also [Seel, 1989, p. 3]).

Clearly, classical logics are not suitable in their standard form for reasoning about intentional notions: alternative formalisms are required. A vast enterprise has sprung up devoted to developing such formalisms.

The field of formal methods for reasoning about intentional notions is widely reckoned to have begun with the publication, in 1962, of Jaakko Hintikka’s book *Knowledge and Belief: An Introduction to the Logic of the Two Notions* [Hintikka, 1962]. At that time, the subject was considered fairly esoteric, of interest to comparatively few researchers in logic and the philosophy of mind. Since then, however, it has become an important research area in its own right, with contributions from researchers in AI, formal philosophy, linguistics and economics. There is now an enormous literature on the subject, and with a major biannual international conference devoted solely to theoretical aspects of reasoning about knowledge, as well as the input from numerous other, less specialized conferences, this literature is growing ever larger.

Despite the diversity of interests and applications, the number of basic techniques in use is quite small. Recall, from the discussion above, that there are two problems to be addressed in developing a logical formalism for intentional notions: a syntactic one, and a semantic one. It follows that any formalism can be characterized in terms of two independent attributes: its *language offormulation*, and *semantic model* [Konolige, 1986, p. 83].

There are two fundamental approaches to the syntactic problem. The first is to use a *modal* language, which contains non-truth-functional *modal operators*, which are applied to formulae. An alternative approach involves the use of a *meta-language*: a many-sorted first-order language containing terms which denote formulae of some other *object language*. Intentional notions can be represented using a meta-language predicate and given whatever axiomatization is deemed appropriate. Both of these approaches have their advantages and disadvantages.

MODAL OPERATOR

META-LANGUAGE

OBJECT LANGUAGE

As with the syntactic problem, there are two basic approaches to the semantic problem. The first, best-known, and probably most widely used approach is to adopt a *possible-worlds* semantics, where an agent’s beliefs, knowledge, goals, etc., are characterized as a set of alternatives, known as possible worlds, with an *accessibility relation* holding between them. Possible-worlds semantics have an associated *correspondence theory* which makes them an attractive mathematical tool to work with [Chellas, 1980]. However, they also have many associated difficulties, notably the well-known logical omniscience problem, which implies that agents are perfect reasoners. A number of minor variations on the possible-worlds theme have been proposed, in an attempt to retain the correspondence theory, but without logical omniscience.

POSSIBLE WORLDS

ACCESSIBILITY RELATION

CORRESPONDENCE THEORY

The most common alternative to the possible-worlds model for belief is to use a *sentential* or *interpreted symbolic structures* approach. In this scheme, beliefs are viewed as symbolic formulae explicitly represented in a data structure associated with an agent. An agent then believes ϕ if ϕ is present in the agent’s belief structure. Despite its simplicity, the sentential model works well under certain circumstances [Konolige, 1986].

INTERPRETED SYMBOLIC STRUCTURES

The next part of this chapter contains detailed reviews of some of these formalisms. First, the idea of possible-worlds semantics is discussed, and then a detailed analysis of normal modal logics is presented, along with some variants on the possible-worlds theme.

17.1.1 Possible-worlds semantics for modal logics

The possible-worlds model for epistemic logics was originally proposed by [Hintikka, 1962], and is now most commonly formulated in a normal modal logic using the techniques developed by [Kripke, 1963]. Hintikka’s insight was to see that an agent’s beliefs could be characterized in terms of a set of *possible worlds*, in the following way. Consider an agent playing the card game gin rummy (this example was adapted from [Halpern, 1987]). In this

game, the more one knows about the cards possessed by one's opponents, the better one is able to play. And yet complete knowledge of an opponent's cards is generally impossible (if one excludes cheating). The ability to play gin rummy well thus depends, at least in part, on the ability to deduce what cards are held by an opponent, given the limited information available. Now suppose our agent possessed the ace of spades. Assuming the agent's sensory equipment was functioning normally, it would be rational of her to believe that she possessed this card. Now suppose she were to try to deduce what cards were held by her opponents. This could be done by first calculating all the various different ways that the cards in the pack could possibly have been distributed among the various players. (This is not being proposed as an actual card-playing strategy, but for illustration!) For argument's sake, suppose that each possible configuration is described on a separate piece of paper. Once the process was complete, our agent could then begin to systematically eliminate from this large pile of paper all those configurations which were *not possible, given what she knows*. For example, any configuration in which she did not possess the ace of spades could be rejected immediately as impossible. Call each piece of paper remaining after this process a *world*. Each world represents one state of affairs considered possible, given what she knows. Hintikka coined the term *epistemic alternatives* to describe the worlds possible given one's beliefs. Something true in *all* our agent's epistemic alternatives could be said to be believed by the agent. For example, it will be true in all our agent's epistemic alternatives that she has the ace of spades.

EPISTEMIC ALTERNATIVES

On a first reading, this technique seems a peculiarly roundabout way of characterizing belief, but it has two advantages. First, it remains neutral on the subject of the cognitive structure of agents. It certainly does not posit any internalized collection of possible worlds. It is just a convenient way of characterizing belief. Second, the mathematical theory associated with the formalization of possible worlds is extremely appealing (see below).

The next step is to show how possible worlds may be incorporated into the semantic framework of a logic. This is the subject of the next section.

17.1.2 Normal modal logics

Epistemic logics are usually formulated as *normal modal logics* using the semantics developed by [Kripke, 1963]. Before moving on to explicitly epistemic logics, this section describes normal modal logics in general.

NORMAL MODAL LOGIC

Modal logics were originally developed by philosophers interested in the distinction between *necessary* truths and mere *contingent* truths. Intuitively, a necessary truth is something that is true *because it could not have been otherwise*, whereas a contingent truth is something that could, plausibly, have been otherwise. For example, it is a fact that as I write, the Labour Party of Great Britain holds a majority in the House of Commons. But although this is true, it is *not* a necessary truth; it could quite easily have turned out that the Conservative Party won a majority at the last general election. This fact is thus only a contingent truth.

NECESSARY CONTINGENT TRUTH

Contrast this with the following statement: *the square root of 2 is not a rational number*

Contrast this with the following statement: the square root of 2 is not a rational number.

There seems no earthly way that this could be anything *but* true (given the standard reading of the sentence). This latter fact is an example of a necessary truth. Necessary truth is usually defined as something true in *all possible worlds*. It is actually quite difficult to think of any necessary truths other than mathematical laws.

To illustrate the principles of modal epistemic logics, I will define a simple normal propositional modal logic. This logic is essentially classical propositional logic, extended by the addition of two operators: ‘ $\Box$ ’ (necessarily), and ‘ $\Diamond$ ’ (possibly).

First, its syntax. Let $Prop = \{p, q, \dots\}$ be a countable set of *atomic propositions*. The syntax of normal propositional modal logic is defined by the following rules.

- 1. If $p \in Prop$, then p is a formula.
- 2. If ϕ, ψ are formulae, then so are

$$\text{true } \neg\phi \ \phi \wedge \psi.$$

- 3. If ϕ is a formula, then so are

$$\Box\phi \ \Diamond\phi$$

The operators ‘ $\neg$ ’ (not) and ‘ $\wedge$ ’ (or) have their standard meanings; true is a logical constant (sometimes called *verum*) that is always true. The remaining connectives of propositional logic can be defined as abbreviations in the usual way. The formula $\Box\phi$ is read ‘necessarily ϕ ’, and the formula $\Diamond\phi$ is read ‘possibly ϕ ’. Now to the semantics of the language.

Normal modal logics are concerned with truth at worlds; models for such logics therefore contain a set of worlds, W , and a binary relation, R , on W , saying which worlds are considered possible relative to other worlds. Additionally, a valuation function V is required, saying what propositions are true at each world.

A model for a normal propositional modal logic is a triple (W, R, V) , where W is a non-empty set of worlds, $R \subseteq W \times W$, and

$$V: W \rightarrow 2^{Prop}$$

is a valuation function, which says for each world $w \in W$ which atomic propositions are true in w . An alternative, equivalent technique would have been to define V as follows:

$$V: W \times Prop \rightarrow \{\text{true}, \text{false}\},$$

though the rules defining the semantics of the language would then have to be changed slightly.

The semantics of the language are given via the satisfaction relation, ‘ $\models$ ’, which holds between pairs of the form (M, w) (where M is a model, and w is a reference world), and formulae of the language. The semantic rules defining this relation are given in [Figure 17.1](#).

The definition of satisfaction for atomic propositions thus captures the idea of truth in the ‘current’ world (which appears on the left of ‘ $\models$ ’). The semantic rules for ‘true’, ‘ $\neg$ ’, and ‘ $\wedge$ ’ are standard. The rule for ‘ $\Box$ ’ captures the idea of truth in all accessible worlds, and the rule for ‘ $\Diamond$ ’ captures the idea of truth in at least one possible world.

Note that the two modal operators are *duals* of each other, in the sense that the universal and existential quantifiers of first-order logic are duals:

$$\Box\phi \Leftrightarrow \neg\Diamond\neg\phi$$

It would thus have been possible to take either one as primitive, and introduce the other as a derived operator.

Figure 17.1: The semantics of normal modal logic.

$\langle M, w \rangle \models$	true	
$\langle M, w \rangle \models$	p	where $p \in Prop$, if and only if $p \in \mathcal{V}(w)$
$\langle M, w \rangle \models$	$\neg\varphi$	if and only if $\langle M, w \rangle \not\models \varphi$
$\langle M, w \rangle \models$	$\varphi \vee \psi$	if and only if $\langle M, w \rangle \models \varphi$ or $\langle M, w \rangle \models \psi$
$\langle M, w \rangle \models$	$\Box\varphi$	if and only if $\forall w' \in W \cdot$ if $(w, w') \in R$ then $\langle M, w' \rangle \models \varphi$
$\langle M, w \rangle \models$	$\Diamond\varphi$	if and only if $\exists w' \in W \cdot (w, w') \in R$ and $\langle M, w' \rangle \models \varphi$

Correspondence theory

To understand the extraordinary properties of this simple logic, it is first necessary to introduce *validity* and *satisfiability*. A formula is

- *satisfiable* if it is satisfied for some model/world pair
- *unsatisfiable* if it is not satisfied by any model/world pair
- *true in a model* if it is satisfied for every world in the model
- *valid in a class of models* if it true in every model in the class
- *valid* if it is true in the class of all models.

If φ is valid, we indicate this by writing $\models \varphi$. Notice that validity is essentially the same as the notion of ‘tautology’ in classical propositional logic – all tautologies are valid.

The two basic properties of this logic are as follows. First, the following axiom schema is valid:

$$\models \Box(\varphi \rightarrow \psi) \rightarrow (\Box\varphi \rightarrow \Box\psi).$$

This axiom is called *K*, in honour of Kripke. The second property is as follows.

$$\text{If } \models \varphi, \text{ then } \models \Box\varphi.$$

Proofs of these properties are trivial, and are left as an exercise for the reader. Now, since *K* is valid, it will be a theorem of any complete axiomatization of normal modal logic.

Similarly, the second property will appear as a rule of inference in any axiomatization of normal modal logic; it is generally called the *necessitation* rule. These two properties turn out to be the most problematic features of normal modal logics when they are used as logics of knowledge/belief (this point will be examined later).

NECESSITATION

The most intriguing properties of normal modal logics follow from the properties of the accessibility relation, *R*, in models. To illustrate these properties, consider the following axiom schema:

$$\Box\varphi \rightarrow \varphi$$

It turns out that this axiom is *characteristic* of the class of models with a *reflexive* accessibility relation. (By *characteristic*, we mean that it is true in all and only those models

accessibility relation. (By characterizing, we mean that it is true in all and only those models in the class.)

Table 17.1: Some correspondence theory.

Name	Axiom	Condition on R	First-order characterization
T	$\Box \phi \rightarrow \phi$	Reflexive	$\forall w \in W \cdot (w, w) \in R$
D	$\Box \phi \rightarrow \Box \Box \phi$	Serial	$\forall w \in W \cdot \exists w' \in W \cdot (w, w') \in R$
4	$\Box \phi \rightarrow \Box \Box \phi$	Transitive	$\forall w, w', w'' \in W \cdot (w, w') \in R \wedge (w', w'') \in R \rightarrow (w, w'') \in R$
5	$\Box \phi \rightarrow \Box \Box \phi$	Euclidean	$\forall w, w', w'' \in W \cdot (w, w') \in R \wedge (w, w'') \in R \rightarrow (w', w'') \in R$

There are a host of axioms which correspond to certain properties of R : the study of the way that properties of R correspond to axioms is called *correspondence theory*. In [Table 17.1](#), I list some axioms along with their characteristic property on R , and a first-order formula describing the property.

A *system of logic* can be thought of as a set of formulae valid in some class of models; a member of the set is called a *theorem* of the logic (if ϕ is a theorem, this is usually denoted by $\vdash \phi$). The notation $K\Sigma_1 \dots \Sigma_n$ is often used to denote the smallest normal modal logic containing axioms $\Sigma_1, \dots, \Sigma_n$ (recall that any normal modal logic will contain K ; cf. [Goldblatt, [1987](#), p. 25]).

For the axioms T , D , 4 , and 5 , it would seem that there ought to be 16 distinct systems of logic (since $2^4 = 16$). However, some of these systems turn out to be equivalent (in that they contain the same theorems), and as a result there are only 11 distinct systems. The relationships between these systems are described in [Figure 17.2](#) (after [Konolige, [1986](#), p. 99] and [Chellas, [1980](#), p. 132]). In this diagram, an arc from A to B means that B is a strict superset of A : every theorem of A is a theorem of B , but not vice versa; $A = B$ means that A and B contain precisely the same theorems.

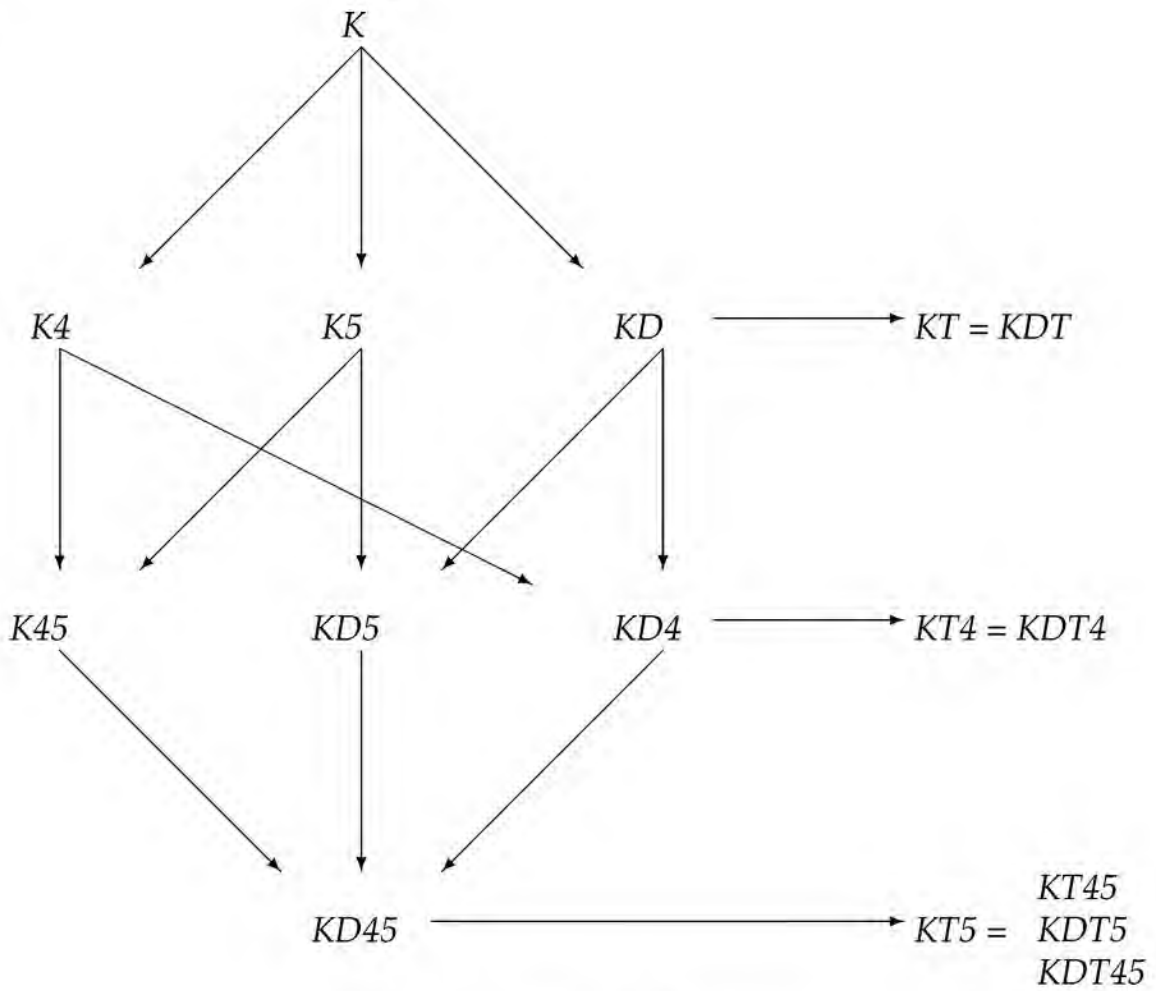
Because some modal systems are so widely used, they have been given names:

KT	is known as T ,
$KT4$	is known as $S4$,
$KD45$	is known as weak- $S5$,
$KT5$	is known as $S5$.

17.1.3 Normal modal logics as epistemic logics

To use the logic developed above as an epistemic logic, the formula $\Box \phi$ is read as ‘it is known that ϕ ’. The worlds in the model are interpreted as epistemic alternatives; the accessibility relation defines what the alternatives are from any given world. The logic deals with the knowledge of a single agent. To deal with multiagent knowledge, one adds to a model structure an indexed set of accessibility relations, one for each agent. A model is then a structure

Figure 17.2: The modal systems based on axioms T , D , 4 and 5 .



$$(W, R_1, \dots, R_n, V),$$

where R_i is the knowledge accessibility relation of agent i . The simple language defined above is extended by replacing the single modal operator ' $\Box$ ' by an indexed set of unary modal operators $\{K_i\}$, where $i \in \{1, \dots, n\}$. The formula $K_i \varphi$ is read ' i knows that φ '. The semantic rule for ' $\Box$ ' is replaced by the following rule:

$$(M, w) \models K_i \varphi \text{ if and only if } \forall w' \in W \cdot \text{if } (w, w') \in R_i \text{ then } (M, w') \models \varphi.$$

Each operator K_i thus has exactly the same properties as ' $\Box$ '. Corresponding to each of the modal systems Σ , above, a corresponding system Σ_n is defined, for the multiagent logic. Thus K_n is the smallest multiagent epistemic logic and $S5_n$ is the largest.

The next step is to consider how well normal modal logic serves as a logic of knowledge/belief. Consider first the necessitation rule and axiom K , since any normal modal system is committed to these.

The necessitation rule tells us that an agent knows all valid formulae. Among other things, this means that an agent knows all propositional tautologies. Since there are an infinite number of these, an agent will have an infinite number of items of knowledge: immediately, one is faced with a counterintuitive property of the knowledge operator.

Now consider the axiom K , which says that an agent's knowledge is closed under implication. Suppose φ is a logical consequence of the set $\Phi = \{\varphi_1, \dots, \varphi_n\}$, then in every world where all of Φ are true, φ must also be true, and hence

$$\varphi_1 \wedge \dots \wedge \varphi_n \rightarrow \varphi$$

must be valid. By necessitation, this formula will also be believed. Since an agent's beliefs are closed under implication, whenever it believes each of Φ , it must also believe φ . Hence an agent's knowledge is closed under logical consequence. This also seems counterintuitive. For example, suppose that, like every good logician, our agent knows Peano's axioms. Now, Fermat's last theorem follows from Peano's axioms – although it took the labour of centuries to prove it. Yet if our agent's beliefs are closed under logical consequence, then our agent must know it. So consequential closure, implied by necessitation and the K axiom, seems an overstrong property for resource-bounded reasoners.

17.1.4 Logical omniscience

These two problems – that of knowing all valid formulae, and that of knowledge/belief being closed under logical consequence – together constitute the famous *logical omniscience* problem. This problem has some damaging corollaries.

LOGICAL OMNISCIENCE

The first concerns consistency. Human believers are rarely consistent in the logical sense of the word; they will often have beliefs φ and ψ , where $\varphi \vdash \neg\psi$, without being aware of the implicit inconsistency. However, the ideal reasoners implied by possible-worlds semantics cannot have such inconsistent beliefs without believing *every* formula of the logical language (because the consequential closure of an inconsistent set of formulae is the set of all formulae). Konolige has argued that logical consistency is much too strong a property for resource-bounded reasoners: he argues that a lesser property, that of being *non-contradictory*, is the most one can reasonably demand [Konolige, 1986]. Non-contradiction means that an agent would not simultaneously believe φ and $\neg\varphi$, although the agent might have logically inconsistent beliefs.

The second corollary is more subtle. Consider the following propositions (this example is from [Konolige, 1986, p. 88]).

1. Hamlet's favourite colour is black.
2. Hamlet's favourite colour is black *and* every planar map can be four coloured.

The second conjunct of (2) is valid, and will thus be believed. This means that (1) and (2) are logically equivalent; (2) is true just when (1) is. Since agents are ideal reasoners, they will believe that the two propositions are logically equivalent. This is yet another counterintuitive property implied by possible-worlds semantics, as 'equivalent propositions are *not* equivalent as beliefs' [Konolige, 1986, p. 88]. Yet this is just what possible-worlds semantics implies. It has been suggested that propositions are thus too *coarse grained* to serve as the objects of belief in this way.

17.1.5 Axioms for knowledge and belief

Let us now consider the appropriateness of the axioms D_n , T_n , 4_n , and 5_n for logics of knowledge/belief.

The axiom D_n says that an agent's beliefs are non-contradictory; it can be rewritten in the following form:

$$K_i\varphi \rightarrow \neg K_i\neg\varphi,$$

which is read ‘if i knows φ , then i does not know $\neg\varphi$ ’. This axiom seems a reasonable property of knowledge/belief.

The axiom T_n is often called the *knowledge axiom*, since it says that what is known is true. It is usually accepted as the axiom that distinguishes knowledge from belief: it seems reasonable that one could believe something that is false, but one would hesitate to say that one could *know* something false. Knowledge is thus often defined as true belief: i knows φ if i believes φ and φ is true. So defined, knowledge satisfies T_n .

Axiom 4_n is called the *positive introspection axiom*. Introspection is the process of examining one’s own beliefs, and is discussed in detail in [Konolige, 1986, Chapter 5]. The positive introspection axiom says that an agent knows what it knows. Similarly, axiom 5_n is the *negative introspection axiom*, which says that an agent is aware of what it does not know. Positive and negative introspection together imply that an agent has perfect knowledge about what it does and does not know (cf. [Konolige, 1986], Equation (5.11), p. 79). Whether or not the two types of introspection are appropriate properties for knowledge/belief is the subject of some debate. However, it is generally accepted that positive introspection is a less demanding property than negative introspection, and is thus a more reasonable property for resource-bounded reasoners.

POSITIVE INTROSPECTION

NEGATIVE INTROSPECTION

Given the comments above, the modal system $S5_n$ is often chosen as a logic of *knowledge*, and weak- $S5_n$ is often chosen as a logic of *belief*.

Discussion

To sum up, the basic possible-worlds approach described above has the following disadvantages as a multiagent epistemic logic:

- agents believe all valid formulae
- agents’ beliefs are closed under logical consequence
- equivalent propositions are identical beliefs
- if agents are inconsistent, then they believe everything.

To which many people would add the following:

[T]he ontology of possible worlds and accessibility relations ... is frankly mysterious to most practically minded people, and in particular has nothing to say about agent architecture.

[Seel, 1989]

Despite these serious disadvantages, possible worlds are still the semantics of choice for many researchers, and a number of variations on the basic possible-worlds theme have been proposed to get around some of the difficulties – see [Wooldridge and Jennings, 1995] for a survey.

17.1.6 Multiagent epistemic logics

Most people, confronted with possible-worlds semantics for the first time, are – initially at least – uncomfortable with the idea:

[The notion] of one possible world being accessible to another has at first sight a certain air of fantasy or science fiction about it.

[Hughes and Cresswell, [1968](#), p. 77]

The problem seems to be with the ontological status of possible worlds: do they really exist? If so, where are they? How do they map onto an agent's physical architecture? If these questions cannot be answered, then one would be reluctant to treat epistemic alternatives as anything other than a theoretical nicety.

Some researchers have proposed *grounding* epistemic alternatives: giving them a precise meaning in the real world, and thus overcoming any confusion about their status. This section describes grounded possible worlds, and will focus on the distributed systems approach; the formal treatment is adapted from [Fagin et al., [1995](#)].

GROUNDING

Using a logic of knowledge to analyse a distributed system may seem strange. However, as Halpern points out, when informally reasoning about a distributed system, one often makes statements such as: 'processor 1 cannot send a packet to processor 2 until it knows that processor 2 received the previous one' [Halpern, [1987](#)]. A logic of knowledge formalizes such reasoning.

The starting point for our study is to define a simple model of distributed systems. A system contains an *environment*, which may be in any of a set E of environment states, and a set of n processes $\{1, \dots, n\}$, each of which may be in any of a set L of 'local' states. At any time, a system may therefore be in any of a set G of global states:

$$G \subseteq E \times \underbrace{L \times \dots \times L}_{n \text{ times}}.$$

Next, a *run* of a system is a function which assigns to each time point a global state: time points are isomorphic to the natural numbers (and time is thus discrete, bounded in the past, and infinite in the future). Note that this is essentially the same notion of runs that was introduced in [Chapter 2](#), but I have formulated it slightly differently. A run r is thus a function $r: \mathbb{N} \rightarrow G$. A *point* is a pair (r, u) consisting of a run together with a time point. Let $Point$ be the set of points of a system. A point implicitly identifies a global state. Points will serve as worlds in the logic of knowledge to be developed. A *system* is a set of runs.

Now, suppose s and s' are two global states.

$$\begin{aligned} s &= (e, l_1, \dots, l_n), \\ s' &= (e', l'_1, \dots, l'_n). \end{aligned}$$

We now define a relation $\sim_i$ on states: for each process i ,

$$s \sim_i s' \text{ if and only if } (l_i = l'_i).$$

Note that $\sim_i$ will be an equivalence relation. The terminology is that if $s \sim_i s'$, then s and s' are *indistinguishable* to i , since the local state of i is the same in each global state. Intuitively, the local state of a process represents the information that the process has, and if two global states are indistinguishable, then it has the same information in each.

The crucial point here is that since a processes [sic] [choice of] actions ... are a function of its local state, if two points are indistinguishable to processor i , then processor i will perform the same actions in each state.

[Halpern, [1987](#), pp. 46–47]

The next step is to define a language for reasoning about such systems. The language is that of the multiagent epistemic logic defined earlier (i.e. classical propositional logic enriched by the addition of a set of unary modal operators K_i , for $i \in \{1, \dots, n\}$). The semantics of the language is presented via the satisfaction relation, ' $\models$ ', which holds between triples of the form (M, r, u) and formula of the language. Here, (r, u) is a point, and M is a structure (R, V) , where R is a system (cf. the set of runs discussed in [Chapter 2](#)), and

$$V: \text{Point} \rightarrow 2^{\text{Prop}}$$

returns the set of atomic propositions true at a point. The structure (R, V) is called an *interpreted system*. The only non-standard semantic rules are for propositions and modal formulae:

INTERPRETED SYSTEM

$(M, r, u) \models p$ where $p \in \text{Prop}$, if and only if $p \in V((r, u))$, $(M, r, u) \models K_i \phi$ if and only if $(M, r', u') \models \phi$ for all $r' \in R$ and $u' \in \mathbf{N}$ such that $r(u) \sim_i r'(u')$.

Note that since $\sim_i$ is an equivalence relation (i.e. it is reflexive, symmetric, and transitive), this logic will have the properties of the system $S5_n$, discussed above. In what sense does the second rule capture the idea of a process's knowledge? The idea is that if $r(u) \sim_i r'(u')$, then *for all i knows*, it could be in either run r , time u , or run r' , time u' ; the process does not have enough information to be able to distinguish the two states. The information/knowledge it *does* have are the things true in all its indistinguishable states.

In this model, knowledge is an *external* notion. We do not imagine a processor scratching its head wondering whether or not it knows a fact ϕ . Rather, a programmer reasoning about a particular protocol would say, from the outside, that the processor knew ϕ because in all global states [indistinguishable] from its current state (intuitively, all the states the processor could be in, for all it knows), ϕ is true.

[Halpern, [1986](#), p. 6]

17.1.7 Common and distributed knowledge

In addition to reasoning about what one agent knows or believes, it is often useful to be able to reason about 'cultural' knowledge: the things that everyone knows, and that everyone knows that everyone knows, etc. This kind of knowledge is called *common knowledge*. The famous 'muddy children' puzzle – a classic problem in epistemic reasoning – is an example of

the kind of problem that is efficiently dealt with via reasoning about common knowledge (see [Fagin et al., 1995] for a statement of the problem).

The starting point for common knowledge is to develop an operator for things that ‘everyone knows’. A unary modal operator E is added to the modal language discussed above; the formula $E\phi$ is read ‘everyone knows ϕ ’. It can be defined as an abbreviation:

$$E\phi \equiv K_1 \phi \wedge \dots \wedge K_n \phi.$$

The E operator does not satisfactorily capture the idea of common knowledge. For this, another derived operator C is required; C is defined, ultimately, in terms of E . It is first necessary to introduce the derived operator E^k ; the formula $E^k \phi$ is read ‘everyone knows ϕ to degree k ’. It is defined as follows:

$$\begin{aligned} E^1 \phi &\equiv E\phi, \\ E^{k+1} \phi &\equiv E(E^k \phi). \end{aligned}$$

The common knowledge operator can then be defined as an abbreviation:

$$C\phi \equiv E\phi \wedge E^2\phi \wedge \dots \wedge E^k\phi \wedge \dots$$

Thus common knowledge is the infinite conjunction: everyone knows, and everyone knows that everyone knows, and so on.

It is interesting to ask when common knowledge can arise in a system. A classic problem in distributed systems folklore is the *coordinated attack problem*.

COORDINATED ATTACK

Two divisions of an army, each commanded by a general, are camped on two hilltops overlooking a valley. In the valley awaits the enemy. It is clear that if both divisions attack the enemy simultaneously they will win the battle, while if only one division attacks, it will be defeated. As a result, neither general will attack unless he is absolutely sure that the other will attack with him. In particular, a general will not attack if he receives no messages. The commanding general of the first division wishes to coordinate a simultaneous attack (at some time the next day). The generals can communicate only by means of messengers. Normally, it takes a messenger one hour to get from one encampment to the other. However, it is possible that he will get lost in the dark or, worse yet, be captured by the enemy. Fortunately, on this particular night, everything goes smoothly. How long will it take them to coordinate an attack?

Suppose a messenger sent by general A reaches General B with a message saying ‘attack at dawn’. Should General B attack? Although the message was in fact delivered, General A has no way of knowing that it was delivered. A must therefore consider it possible that B did not receive the message (in which case B would definitely not attack). Hence A will not attack given his current state of knowledge. Knowing this, and not willing to risk attacking alone, B cannot attack solely based on receiving A ’s message. Of course, B can try to improve matters by sending the messenger back to A with an acknowledgment. When A receives this acknowledgment, can he then attack? A here is in a similar position to the one B was in when he received the original message. This time B does not know that the acknowledgment was delivered.

Intuitively, the two generals are trying to bring about a state where it is common knowledge between them that the message to attack was delivered. Each successive round of communication, even if successful, only adds one level to the depth of nested belief. *No* amount of communication is sufficient to bring about the infinite nesting that common knowledge requires. As it turns out, if communication delivery is not guaranteed, then common knowledge can *never* arise in such a scenario. Ultimately, this is because, no matter how many messages and acknowledgments are sent, at least one of the generals will always be uncertain about whether or not the last message was received.

One might ask about whether *infinite* nesting of common knowledge is required. Could the two generals agree between themselves beforehand to attack after, say, only two acknowledgments? Assuming that they could meet beforehand to come to such an agreement, then this would be feasible. But the point is that whoever sent the last acknowledgment would be uncertain as to whether this was received, and would hence be attacking while unsure as to whether it was a coordinated attack or a doomed solo effort.

A related issue to common knowledge is that of distributed, or implicit, knowledge. Suppose there is an omniscient observer of some group of agents, with the ability to ‘read’ each agent’s beliefs/knowledge. Then this agent would be able to pool the collective knowledge of the group of agents, and would generally be able to deduce more than any one agent in the group. For example, suppose, in a group of two agents, agent 1 only knew ϕ , and agent 2 only knew $\phi \rightarrow \psi$. Then there would be *distributed* knowledge of ψ , even though no agent explicitly knew ψ . Distributed knowledge cannot be reduced to any of the operators introduced so far: it must be given its own definition. The distributed knowledge operator D has the following semantic rule:

DISTRIBUTED KNOWLEDGE

$(M, w) \models D\phi$ if and only if $(M, w') \models \phi$ for all w' such that $(w, w') \in (R_1 \cap \dots \cap R_n)$.

This rule might seem strange at first, since it uses set intersection rather than set union, which is at odds with a naive perception of how distributed knowledge works. However, a *restriction* on possible worlds generally means an *increase* in knowledge.

Distributed knowledge is potentially a useful concept in cooperative problem-solving systems, where knowledge about a problem is distributed among a group of problem-solving agents, which must try to deduce a solution through cooperative interaction.

The various group knowledge operators form a hierarchy:

$$C\phi \rightarrow E^k \phi \rightarrow \dots \rightarrow E\phi \rightarrow K_i \phi \rightarrow D\phi.$$

See [Fagin et al., 1995] for further discussion of these operators and their properties.

17.2 Logics for Mental States

So far, we have considered only logics that attempt to characterize the information (knowledge or beliefs) that agents have. Although some interesting properties may be expressed using such logics, to capture many multiagent scenarios, we require some way of characterizing the preferences of agents, for example in the form of their goals or desires. An obvious approach to

developing a logic of goals or desires is to adapt possible-worlds semantics. In this view, each goal-accessible world represents one way the world might be if the agent’s goals were realized. However, this approach falls prey to what is called the *side effect* problem: it predicts that agents have a goal of the logical consequences of their goals (cf. the logical omniscience problem, discussed above). This is not a desirable property: one might have a goal of going to the dentist, with the necessary consequence of suffering pain, without having a goal of suffering pain. The problem is discussed (in the context of intentions) in [Bratman, 1990].

17.2.1 Cohen and Levesque’s intention logic

One of the best-known, and most sophisticated, attempts to show how the various components of an agent’s cognitive make-up could be combined to form a logic of rational agency is due to [Cohen and Levesque, 1990a]. Cohen and Levesque’s formalism was originally used to develop a theory of intention (as in ‘I intended to...’), which the authors required as a prerequisite for a theory of speech acts (see [Section 17.2.2](#) for a summary, and [Cohen and Levesque, 1990b] for full details). This section will focus on the use of the logic in developing a theory of intention. The first step is to lay out the criteria that a theory of intention must satisfy.

When building intelligent agents – particularly agents that must interact with humans – it is important that a *rational balance* is achieved between the beliefs, goals, and intentions of the agents.

RATIONAL BALANCE

For example, the following are desirable properties of intention: an autonomous agent should act on its intentions, not in spite of them; adopt intentions it believes are feasible and forego those believed to be infeasible; keep (or commit to) intentions, but not forever; discharge those intentions believed to have been satisfied; alter intentions when relevant beliefs change; and adopt subsidiary intentions during plan formation.

[Cohen and Levesque, 1990a, p. 214]

Recall the properties of intentions, as discussed in [Chapter 4](#).

1. Intentions pose problems for agents, who need to determine ways of achieving them.
2. Intentions provide a ‘filter’ for adopting other intentions, which must not conflict.
3. Agents track the success of their intentions, and are inclined to try again if their attempts fail.
4. Agents believe their intentions are possible.
5. Agents do not believe that they will not bring about their intentions.

Table 17.2: Atomic modalities in Cohen and Levesque’s logic.

Operator	Meaning
(Bel i ϕ)	agent i believes ϕ
(Goal i ϕ)	agent i has a goal of ϕ
(Happens α)	action α will happen next
(Done α)	action α has just happened

6. Under certain circumstances, agents believe that they will bring about their intentions.

6. Under certain circumstances, agents believe that they will bring about their intentions.

7. Agents need not intend all the expected side effects of their intentions.

Given these criteria, Cohen and Levesque adopt a two-tiered approach to the problem of formalizing a theory of intention. First, they construct the logic of rational agency, 'being careful to sort out the relationships among the basic modal operators' [Cohen and Levesque, 1990a, p. 221]. On top of this framework, they introduce a number of derived constructs, which constitute a 'partial theory of rational action' [Cohen and Levesque, 1990a, p. 221]; intention is one of these constructs.

Syntactically, the logic of rational agency is a many-sorted, first-order, multi-modal logic with equality, containing four primary modalities (see [Table 17.2](#)).

The semantics of Bel and Goal are given via possible worlds, in the usual way: each agent is assigned a belief accessibility relation, and a goal accessibility relation. The belief accessibility relation is Euclidean, transitive, and serial, giving a belief logic of KD45. The goal relation is serial, giving a conative logic KD. It is assumed that each agent's goal relation is a subset of its belief relation, implying that an agent will not have a goal of something it believes will not happen. Worlds in the formalism are a discrete sequence of events, stretching infinitely into past and future.

The two basic temporal operators, Happens and Done, are augmented by some operators for describing the structure of event sequences, in the style of dynamic logic [Harel et al., 2000]. The two most important of these constructors are '?', and '?:'

α , α' denotes α followed by α' , $\phi?$ denotes a 'test action' ϕ .

The standard future time operators of temporal logic, ' $\square$ ' (always), and ' $\square$ ' (sometimes), can be defined as abbreviations, along with a 'strict' sometime operator, Later:

$\square \alpha \equiv \exists x \cdot (\text{Happens } x; \alpha?)$,	$\square \alpha \equiv \neg \square \neg \alpha$	$(\text{Later } p) \equiv \neg p \wedge \square p.$
--	--	---

A temporal precedence operator, (Before p q), can also be derived, and holds if p holds before p . An important assumption is that *all* goals are eventually dropped:

$$\square \neg (\text{Goal } x (\text{Later } p)).$$

The first major derived construct is a *persistent* goal:

$$(\text{P-Goal } i \ p) \equiv (\text{Goal } i \ (\text{Later } p)) \wedge (\text{Bel } i \ \neg p) \wedge \left[\begin{array}{l} \text{Before} \\ ((\text{Bel } i \ p) \vee (\text{Bel } i \ \square \neg p)) \\ \neg (\text{Goal } i \ (\text{Later } p)) \end{array} \right].$$

So, an agent has a persistent goal of p if

1. it has a goal that p eventually becomes true, and believes that p is not currently true, and
2. before it drops the goal, one of the following conditions must hold:

(a) the agent believes the goal has been satisfied

(b) the agent believes the goal will never be satisfied.

It is a small step from persistent goals to a first definition of intention, as in ‘intending to act’. Note that ‘intending that something becomes true’ is similar, but requires a slightly different definition (see [Cohen and Levesque, 1990a]). An agent i intends to perform action α if it has a persistent goal to have brought about a state where it had just believed it was about to perform α , and then did α :

$$(\text{Int } i \alpha) = (\text{P-Goal } i [\text{Done } i (\text{Bel } i (\text{Happens } \alpha))]; \alpha).$$

Cohen and Levesque go on to show how such a definition meets many of Bratman’s criteria for a theory of intention (outlined above). In particular, by basing the definition of intention on the notion of a *persistent goal*, Cohen and Levesque are able to avoid overcommitment or undercommitment. An agent will only drop an intention if it believes that the intention has either been achieved, or is unachievable.

17.2.2 Modelling speech acts

We saw in earlier chapters how speech acts form the basis of communication in most multiagent systems. Using their logic of intention, Cohen and Levesque developed a theory which arguably represents the state of the art in the logical analysis of speech acts [Cohen and Levesque, 1990b]. Their work proceeds from two basic premises.

1. Illocutionary force recognition is unnecessary.

What speakers and hearers have to do is only recognize each other’s intentions (based on mutual beliefs). We do not require that those intentions include intentions that the hearer recognize precisely what illocutionary act(s) were being performed.

[Cohen and Levesque, 1990b, p. 223]

2. Illocutionary acts are *complex event types*, and not primitives.

Given this latter point, one must find some way of describing the actions that are performed. Cohen and Levesque’s solution is to use their logic of rational action, which provides a number of primitive event types, which can be put together into more complex event types, using dynamic-logic-style constructions. Illocutionary acts are then defined as complex event types.

Their approach is perhaps best illustrated by giving their definition of a request. Some preliminary definitions are required. First, *alternating belief*:

$$(\text{A-Bel } n \ x \ y \ p) \equiv \underbrace{(\text{Bel } x (\text{Bel } y (\text{Bel } x \cdots (\text{Bel } x \ p) \cdots))}_{n \text{ times}} \underbrace{)}_{n \text{ times}}.$$

And the related concept of *mutual belief*:

$$(\text{M-Bel } x \ y \ p) = \forall n (\text{A-Bel } n \ x \ y \ p).$$

Next, an *attempt* is defined as a complex action expression – hence the use of curly brackets, to distinguish it from a predicate or modal operator:

$$\left[(\text{Bel } x \ \neg p) \quad \wedge \quad \right]$$

$$\{ \text{Attempt } x e p q \} \equiv \left[\begin{array}{l} (\text{Goal } x(\text{Happens } x e; p?)) \wedge \\ (\text{Int } x e; p?) \end{array} \right] ?; e.$$

In English:

An attempt is a complex action that agents perform when they do something (e) desiring to bring about some effect (p) but with intent to produce at least some result (q).

[Cohen and Levesque, [1990b](#), p. 240]

The idea is that p represents the ultimate goal that the agent is aiming for by doing e ; the proposition p represents what it takes to at least make an ‘honest effort’ to achieve p . A definition of *helpfulness* is now presented:

$$(\text{Helpful } x y) \equiv \forall e. \left[\begin{array}{l} (\text{Bel } x (\text{Goal } y \sqsupset (\text{Done } x e))) \wedge \\ \neg (\text{Goal } x \sqsupset \neg (\text{Done } x e)) \\ \rightarrow (\text{Goal } x \sqsupset \neg (\text{Done } x e)). \end{array} \right]$$

In English:

[C]onsider an agent $[x]$ to be helpful to another agent $[y]$ if, for any action $[e]$ he adopts the other agent’s goal that he eventually do that action, whenever such a goal would not conflict with his own.

[Cohen and Levesque, [1990b](#), p. 230]

The definition of requests can now be given (note again the use of curly brackets: requests are complex event types, not predicates or operators):

$$\begin{aligned} \{ \text{Request } spkr addr e \alpha \} \equiv & \{ \text{Attempt } spkr e \phi \\ & (\text{M-Bel } addr spkr (\text{Goal } spkr \phi)) \\ & \} \\ & \text{where } \phi \text{ is} \\ & \sqsupset (\text{Done } addr \alpha) \wedge \\ & (\text{Int } addr \alpha \\ & \left[\begin{array}{l} (\text{Goal } spkr \sqsupset (\text{Done } addr \alpha)) \wedge \\ (\text{Helpful } addr spkr) \end{array} \right] \\ &). \end{aligned}$$

In English:

A request is an attempt on the part of $spkr$, by doing e , to bring about a state where, ideally, (i) $addr$ intends α (relative to the $spkr$ still having that goal, and $addr$ still being helpfully inclined to $spkr$), and (ii) $addr$ actually eventually does α , or at least brings about a state where $addr$ believes that it is mutually believed that it wants the ideal situation.

By this definition, there is no primitive request act:

[A] speaker is viewed as having performed a request if he executes any sequence of actions that produces the needed effects.

[Cohen and Levesque, [1990b](#), p. 246]

In short, any event, of whatever complexity, that satisfies this definition, can be counted as a request. Cohen and Levesque show that if a request takes place, it is possible to infer that many of Searle's preconditions for the act must have held [Cohen and Levesque, [1990b](#), pp. 246–251].

Using Cohen and Levesque's work as a starting point, Galliers developed a more general framework for multiagent dialogue, which acknowledges the possibility for conflict [Galliers, [1988b](#)].

17.3 Logics for Cooperation

Temporal logics have been perhaps the most successful formalism in the specification and verification of conventional reactive and distributed systems [Emerson, [1990](#)], and the associated verification technology of *model checking* for temporal logics has proven to be enormously successful [Clarke et al., [2000](#); Holzmann, [2003](#)]. However, conventional temporal logics are not well suited for expressing the properties of economic, game-like systems.

Work over the past half decade has focused on the use of *cooperation logics* for automated mechanism design and analysis. Cooperation logics were developed independently and more-or-less simultaneously by several researchers in the late 1990s [Alur et al., [1997](#); Pauly, [2002](#)]. As we shall see, although cooperation logics are in fact descended from conventional temporal logics, they are ideal for expressing the strategic properties of systems.

COOPERATION LOGICS

Alternating-time temporal logic (ATL) is perhaps the best-known cooperation logic. ATL emerged from the use of computation tree logic (CTL) for the specification and verification of reactive systems [Emerson, [1990](#)]. CTL is a temporal logic that is interpreted over treelike structures, in which nodes represent time points and arcs represent transitions between time points. In distributed/reactive systems applications, the set of all paths through a tree structure is interpreted as the set of all possible computations of a system. CTL combines *path quantifiers* 'A' and 'E' for expressing that a certain series of events will happen on all paths and on some path respectively, with *tense modalities* for expressing that something will happen eventually on some path ($\Box$), always on some path ($\Diamond$), and so on. Thus, for example, by using CTL-like logics, one may express properties such as 'on all possible computations, the system never enters a fail state', which is represented by the CTL formula

ATL

$$A \Box \neg fail.$$

Although they have proven to be enormously useful in the specification and verification of reactive systems [Clarke et al., [2000](#)], logics such as CTL are of limited value for reasoning about systems in which strategic behaviour is of concern. The kinds of properties we wish to express of such systems typically relate to the *strategic powers* that those system components

have. For example, we might wish to express the fact that ‘agents 1 and 2 can cooperate to ensure that, no matter what agents 3 and 4 do, the system never enters a fail state’. It is not possible to capture such statements using CTL-like logics. The best one can do is either state that something will inevitably happen, or else that it may possibly happen: CTL-like logics thus have no notion of agency. Alur, Henzinger, and Kupferman developed ATL in an attempt to remedy this deficiency. The key insight in ATL is that path quantifiers can be replaced by *cooperation modalities*: the ATL expression

STRATEGIC POWERS

COOPERATION MODALITIES

$$\langle\langle C \rangle\rangle\varphi,$$

where C is a group of system components (agents), expresses the fact that C can cooperate to ensure that, no matter how other system components behave, φ will result. Thus

$$\langle\langle C \rangle\rangle\varphi,$$

captures the *strategic ability of C to bring about φ* . So, for example, the fact that ‘agents 1 and 2 can ensure that the system never enters a fail state, no matter what agents 3 and 4 do’ may be captured in ATL by the following formula:

$$\langle\langle 1,2 \rangle\rangle \Box \neg fail.$$

Using ATL, it is possible to naturally capture interesting strategic properties of multiagent systems. For example, the fact that i can ensure that eventually j has the ability to achieve his goal can be expressed as:

$$\langle\langle i \rangle\rangle \Box \langle\langle j \rangle\rangle \Box goal_j.$$

The fact that one agent is *dependent* on another can be represented by the predicate $VETO(i, j)$, meaning ‘ j needs i to achieve his goal’:

$$VETO(i, j) \equiv \bigwedge_c \langle\langle C \rangle\rangle \Box goal_j \rightarrow \neg \langle\langle C \setminus \{i\} \rangle\rangle \Box goal_j.$$

It is not difficult to see how we can use ATL to define interesting strategic properties of systems. However, ‘raw’ ATL lacks many features required to characterize *gamelike* systems. We will now see some extensions to ATL that attempt to overcome these limitations.

17.3.1 Incomplete information

Incomplete information plays a role in most multiagent scenarios. For example, in a sealed bid auction, the fact that I do not know what you are bidding (and you do not know what I am bidding) is an essential aspect of the mechanism. It is therefore very natural to consider *epistemic* extensions to ATL [van der Hoek and Wooldridge, 2003b]. The idea is to add epistemic modalities K_i for each agent i to ATL: a formula $K_i\varphi$ is intended to express the fact that agent i knows φ . The resulting language, ATEL, is extremely powerful and very natural for expressing the properties of communicating systems. For example, the following formula expresses that a can communicate its knowledge of φ to b :

$$K_a\varphi \rightarrow \langle\langle a \rangle\rangle \Box K_b\varphi.$$

As another example, consider a security protocol, in which agents a and b share some common secret (a key S_{ab} for instance); what one typically wants is the following, which expresses that a can send private information to b , without revealing the message to another agent c :

$$K_a\varphi \wedge \neg K_b\varphi \wedge \neg K_c\varphi \wedge \langle\langle a, b \rangle\rangle \circ (K_a\varphi \wedge K_b\varphi \wedge \neg K_c\varphi).$$

Knowledge preconditions, of the type introduced to the planning community by Moore [Moore, 1990], are also very naturally expressed in ATEL. The fact that knowledge of ψ is a necessary precondition to be able to achieve φ is represented by the following.

$$\langle\langle a \rangle\rangle \circ \varphi \rightarrow K_a\psi.$$

Of course, the interaction between knowledge and ability is rather complex and subtle, and some of the issues raised by Moore are reviewed in [Jamroga and van der Hoek, 2004]. A detailed case study, in which we show how epistemic-ability properties may be model checked, is given in [van der Hoek and Wooldridge, 2003a].

17.3.2 Cooperation logics for social choice

One of the most interesting applications of ATL is in the specification of *social choice procedures*, of the type discussed in Chapter 12. This idea was proposed by Marc Pauly, whose insight was that the ATL cooperation modality construct can be used to express the desirable properties of social choice mechanisms. To see how this works, consider the following informal requirements for a simple social choice mechanism [Pauly, 2001]:

Two individuals, A and B , must choose between two outcomes, p and q . We want a mechanism that will allow them to choose, which will satisfy the following requirements: We want an outcome to be possible – that is, we want the two agents to choose, collectively, either p or q . We do not want them to be able to bring about both outcomes simultaneously. Finally, we do not want either agent to be able to unilaterally dictate an outcome – we want them both to have ‘equal power’.

These requirements may be formally and naturally represented using ATL, as follows:

$$\langle\langle A, B \rangle\rangle \circ p \tag{17.6}$$

$$\langle\langle A, B \rangle\rangle \circ q \tag{17.7}$$

$$\neg \langle\langle A, B \rangle\rangle \circ (p \wedge q) \tag{17.8}$$

$$\neg \langle A \rangle \circ p \tag{17.9}$$

$$\neg \langle B \rangle \circ p \tag{17.10}$$

$$\neg \langle A \rangle \circ q \tag{17.11}$$

$$\neg \langle B \rangle \circ q \tag{17.12}$$

Property (17.6) states that A and B can collectively choose p , while (17.7) states that they can choose q ; (17.8) states that they cannot choose p and q simultaneously; and properties

choose q , (17.8) states that they cannot choose p and q simultaneously, and properties (17.9)–(17.12) state that neither agent can dictate an outcome.

This approach – specifying the desirable properties of a mechanism using such a logic – is the *logic for automated mechanism design and analysis* paradigm, of which the first contours were sketched in [Pauly and Wooldridge, 2003].

If you read [Chapter 12](#), it should be clear that we are not too far away from the kinds of properties that Arrow and Gibbard-Satterthwaite deal with in their famous theorems. However, we can only *explicitly* capture properties such as dictatorship using ‘standard’ ATL.

17.4 Putting Logic to Work

So far, we have said little about how the various logics discussed in this chapter might actually be *used*. We will therefore now look at the role that logic plays in engineering systems.

17.4.1 Logic in specification

In this section, we consider the problem of *specifying* an agent system. A specification for a system can be thought of as a definition of the properties that the system should exhibit. Among other things, a specification defines what the system is intended to do. Typically in software development, a specification forms a key part of the contract between the software client (the person who wants the software) and those who will design and implement the software. There are many arguments for using formal languages of one sort or another to specify systems (e.g., see [Jones, 1990]): the fact that formal specifications are unambiguous is one; the fact that formal specifications are amenable to formal proof is another.

A logical specification can be thought of simply as a formula ϕ . The idea is that such a specification expresses the desirable behaviour of a system. To see how this might work, consider the following, intended to form part of a specification of a process control system:

if

i believes valve 32 is open

then

i should intend that j should believe valve 32 is open.

Expressed in a Cohen–Levesque type logic, this statement becomes the formula:

$$(\text{Bel } i \text{ Open}(\text{valve32})) \rightarrow (\text{Int } i (\text{Bel } j \text{ Open}(\text{valve32}))).$$

It should be intuitively clear how a system specification might be constructed using such formulae, to define the intended behaviour of a system.

One of the main desirable features of a software specification language is that it should not dictate *how* a specification will be satisfied by an implementation. The specification above has exactly this property: it does not dictate how agent i should go about making j aware that valve 32 is open. We simply expect i to behave as a rational agent would, given such an intention [Wooldridge, 2000b].

There are a number of problems with the use of such languages for specification. The most worrying of these is with respect to their semantics. The semantics for the modal connectives (for beliefs, desires, and intentions) is given in the normal modal logic tradition of possible worlds [Chellas, 1980]. So, for example, an agent’s beliefs in some state are characterized by a set of different states, each of which represents one possibility for how the world could